RED ALERT 2 / YURI'S REVENGE

THE GAME ENGINE EXPLAINED

Illustrated reading guide



Read the decisions behind the battlefield.

Plain-language workflows, original data and actual project captures.

157 mobile types / 408 structures / 38 story missions

Plus tutorials, mission logic, AI plans, sources and study exercises.

Prepared 26 October 2026 | Source-grounded learning edition

Cover capture: historical Android-emulator Time Lapse startup; not a full playthrough.

How to use this edition

Read this volume for the conceptual engine, selected unit and structure mechanisms, all campaign workflows and visual examples. The companion Complete Engine Reference retains the entire Markdown guide, including every registered type card and every detailed scenario trigger, roster, script and override. Selected overviews here are explicitly abridged; the complete reference is not.

Screenshots are existing project captures, with their provenance and limits. Workflow drawings are authored explanations. Placement schematics show initial objects from local maps; terrain, elevation, passability and later reinforcements are not reconstructed in them.

Confirmed data, binary-supported findings, conceptual reconstructions and untested branches retain the distinctions in the source guide. No complete campaign playthrough or physical Samsung qualification was added for this export.

Blue links jump to a section included in this PDF. A target not present in the abridged reading edition opens the matching GitHub source page. The complete reference resolves its guide-document cross-references internally.

Each mission schematic uses owner colors as identifiers, not as an unproved friend/enemy classification. Square markers are structures, dot markers are mobile objects, and numbered markers identify notable type records.

Open the PDF bookmarks panel for chapters, mission pages and catalogue sections. Search by display name or stable ID when several objects share the same name.

Contents

Red Alert 2 and Yuri's Revenge: a non-code engine guide 4

How the engine works, one decision at a time 6

Units: illustrated behavior walkthroughs 11

Combat, money, power, and special effects 17

Structures: illustrated service walkthroughs 23

Building a base and reading its dependencies 27

How the computer army and mission scripts cooperate 31

Every campaign, explained as a workflow 34

How the Android version connects to the original game 77

Reading while playing: small engine experiments 81

Images, screenshots and their provenance 85

Sources, verification and limits 91

Red Alert 2 and Yuri's Revenge: a non-code engine guide

Read the game as a collection of decisions: what happens first, what is checked, what changes, and what happens next. This guide connects the original binaries, game rules, campaign maps, supplied manuals, and the local Android implementation.

It covers **157 distinct registered mobile types**, **408 distinct registered structure types**, **38 story missions**, both Boot Camp missions, and one additional legacy/alternate map. Detailed mission references translate **4,274 triggers**, **4,787 event records**, and **13,724 ordered actions**. Separate references explain **341 game-specific weapon/effect paths** and **266 global AI team templates**. See Sources and limits for exactly what these counts mean.

Start here

1. How the engine works - startup, live objects, commands, movement, targeting, production, missions, saves and rendering.
2. Every mobile piece - step-by-step unit behavior and separate RA2/Yuri properties, including hidden, civilian and managed attacker records.
3. Combat and economy - weapons, armor, special effects, resources, money, power, repairs and superweapons.
4. Every establishment and structure - construction, factories, defenses, technology, income, repair, landmarks, scenery and mission-only additions.
5. All campaigns, step by step - the intended progression of each mission, with links to its complete logic ledger.
6. Computer-army workflow - houses, task forces, recruitment, teams, scripts, staging, reinforcements and alliances.
7. Android workflow - native libraries, host services, touch/pen/mouse adaptation, save names and returning to the chooser.
8. Reading while playing - small observation exercises that connect the explanations to your own games.
9. Building a base and reading prerequisites - faction workflows, production dependencies and a plain-language glossary for the reference cards.

Read chapters 1, 3, 5 and 6 first for the overall engine. Treat the large unit, structure, weapon and mission pages as a reference library, rather than trying to memorize them in order. All links are relative so the folder remains readable after downloading it.

Questions you can follow through the book

Question	Reading path
Why does the same IFV behave differently?	IFV, then its passenger's IFV mode and linked weapon
Why does my Chrono Miner drive outward but return differently?	Chrono Miner, then economy
How does the Slave Miner become a building?	Mobile miner, deployed miner
Why does a Robot Tank stop when power fails?	Robot Tank, control center
Why did the mission not end after I destroyed the main base?	Its campaign workflow, then terminal gates and incoming trigger links
Why does a landmark matter even without a special weapon?	Its structure card, then attached tags in the mission ledger
Where did an enemy wave come from without a factory?	Create Team versus Reinforcement, then the mission's team plan
How does my finger become an original mouse command?	Android input conversion

How to read the evidence

- **Data-confirmed** means the value, definition or relationship is present in the extracted local data.
- **Binary-supported** means the explanation also has identified local binary/lifted-procedure evidence.
- **Manual-described** means the supplied manual describes that player-visible behavior.
- **Conceptual reconstruction** means a readable decision sequence, without claiming every original instruction has been recovered.
- **Unverified in play** means the prediction/branch was not demonstrated by a recorded playthrough.

The narrative workflows combine these sources, and their limits are stated where they matter. The detailed ledgers are static translations of declared logic. A declared trigger may be disabled, difficulty-specific, unused, or dependent on an event that never occurs on a particular route. A unit being registered is not proof it can be bought in a normal battle.

No complete campaign playthrough was performed to write this book. Existing emulator QA establishes particular startup, input and save/reload cases. Exact pathfinder internals, every hardcoded default, all target-score formulas and every mission branch remain outside the verified claims. The guide keeps those gaps visible instead of filling them with invented algorithms.

Reference shelves

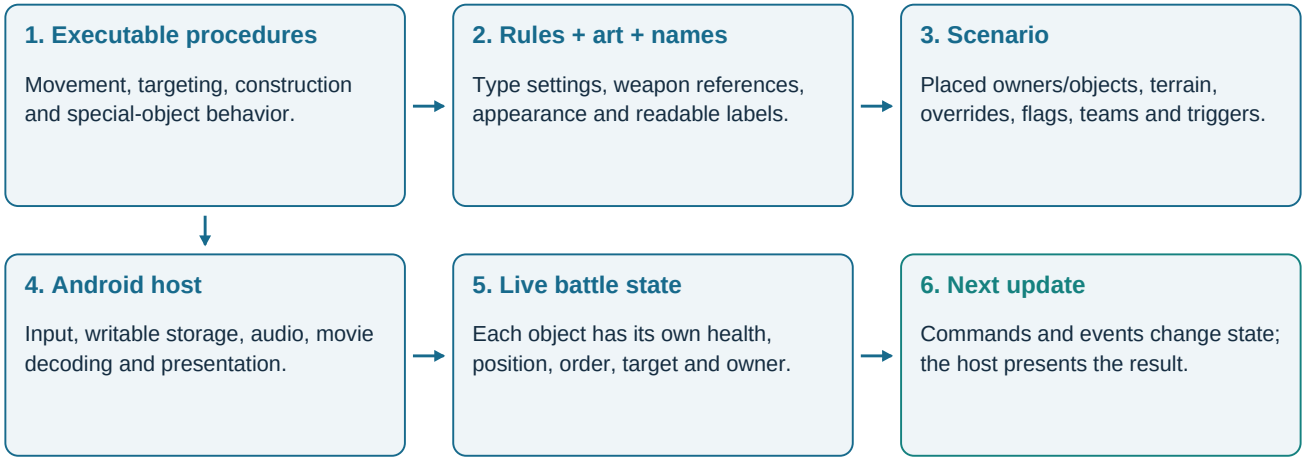
- Weapon, projectile and warhead paths
- Global AI rosters and ordered plans
- Every mission's trigger ledger, team-plan companion and override companion, linked from Campaigns
- Sources, fingerprints, coverage and limitations

The reading folder contains prose and factual reference tables. You do not need a compiler, reverse-engineering tool or programming knowledge to use it. Technical IDs in parentheses are stable cross-references that distinguish objects sharing the same display name.

How the engine works, one decision at a time

Reading order and evidence labels | Units | Buildings | Missions

Four layers behind one battle



Conceptual dependency diagram. It does not assert an exact instruction order within every original frame.

The four layers behind a live battle. Explanatory diagram derived from the guide and local data; not a screenshot.

This chapter explains the original RA2 and Yuri engine in ordinary language. A unit is a soldier, vehicle, ship, aircraft, or other mobile object. A structure is a building, defense, landmark, or map prop. A house is an owner: a player, enemy army, civilian group, or scripted faction. An algorithm is simply a repeatable sequence of decisions.

1. The four things that cooperate

Layer	What it contributes	Example
Original executable	General procedures and special behavior	How a vehicle moves, how an Engineer enters a building, how a trigger fires
Rules and art data	Settings and references	Tank health, weapon selection, prerequisites, model and animation
Scenario map	A particular battle's starting state and instructions	Which base exists, where Tanya starts, which bridge matters
Android host	Phone/tablet services and input adaptation	Two-finger right click, file access, video playback, presenting the frame

Confirmed from local files: the delivered app has separate recompiled RA2 and Yuri libraries. It preserves the original games' procedures while replacing required operating-system services. It is therefore essential to select the correct rules and campaign, rather than treating Yuri as a collection of extra units added to the RA2 rules. See sources.

2. What happens after you choose a game

1. The launcher identifies RA2 or Yuri's Revenge.
2. The host prepares that game's writable working directory and the required packaged assets.
3. The corresponding native library initializes the original program's memory and operating-system connections.
4. The game locates its archives. An archive is a container holding named resources; it is not itself the game simulation.
5. Rules describe available object types. Art records describe their appearance. String tables supply readable names and messages.

6. A campaign choice identifies a starting map. The campaign progression table identifies later stages.
7. The map supplies terrain, houses, starting objects, waypoints, tags, variables, teams, and local rule changes.
8. The engine creates live objects from those definitions and begins the scenario.

Important finding: some maps contain old-looking next-scenario fields that disagree with the campaign progression table. The mission catalogue follows the progression table and mission descriptions, rather than assuming every old field is an active campaign route.

3. A definition is different from a live object

A Rhino's definition is a recipe shared by all Rhinos in that ruleset. A particular Rhino on the battlefield has its own owner, current health, position, facing, order, target, and experience.

1. Read the shared Rhino recipe.
2. Create a new battlefield instance.
3. Give that instance its owner, starting position and starting condition.
4. Change the instance as it moves, fires or takes damage.
5. Leave other Rhinos alone unless the relevant rule affects them too.

This explains why two identical-looking tanks can have different health or rank. It also explains why a campaign can change all Rhinos in that scenario without changing the global skirmish recipe. A map's explicit overrides take precedence for the properties it replaces; unspecified properties still come from the effective rules.

4. One simulation step

Conceptual reconstruction: the following is a useful dependency order for learning. It is not a claim that every original function executes in exactly this order during every frame.

1. Collect pending commands and relevant events.
2. Decide which live objects can act: alive, present, enabled, and in a state that permits the action.
3. Advance movement and orders. Check whether a destination, target, transport or building is still valid.
4. Advance weapons, projectiles, special effects and recovery timers.
5. Apply damage, ownership changes, construction progress, money and power changes as their procedures require.
6. Notify mission logic about events such as entry, discovery, destruction or a changed flag.
7. Evaluate eligible mission conditions and execute their actions.
8. Update the visible battlefield, sidebar, sound and movies from the resulting state.
9. Repeat until paused, a mission outcome occurs, or the game exits.

Simulation time and display time are different. A smooth-looking display does not prove a different movement or weapon rule. Frame counts in data must not automatically be read as seconds on the wall clock. Pausing and changing game speed affect the relationship.

5. What a click means

1. Convert the physical input into the game's mouse coordinates and button state.
2. Determine what is under the cursor: interface control, terrain cell, unit, building, or empty space.
3. Consider the current selection and any modifier keys.
4. Ask what action those objects support. An Engineer may enter and capture; a tank may attack; a miner may harvest.
5. Show an appropriate cursor when the interface has enough information.
6. Queue or apply the selected order through the game's normal command path.
7. The object's later updates carry out the order; the click itself does not teleport it to completion.

Right click is contextual too. A short right click cancels/deselects in the normal interface. Holding right and moving pans the view. Panning changes the camera, not the position of the army. Android adaptation explains how touch and pen reach these same actions.

6. Movement: request, route, progress, reconsider

1. Establish a destination, or choose a position from which to interact with a target.
2. Check the object's movement type and the terrain it can use.
3. Consider cells, occupied spaces, bridges, ramps and other obstructions relevant to that movement type.
4. Find a usable route under those constraints.
5. Let the object's movement controller advance along it, turning and accelerating as applicable.
6. If something blocks the next part, wait, adjust, or request another route as the relevant procedure decides.
7. Stop or transition to another order when close enough, when the target disappears, or when a replacement command arrives.

Boundary: the exact search algorithm, tie-breaking rules and every movement exception have not been proved here. Calling the original pathfinder a particular textbook algorithm without tracing its routines would overstate the evidence. The extracted data does confirm different movement controllers and permitted zones. A submarine, hover transport, aircraft and ordinary tank do not share identical movement constraints.

An apparent pathfinding error can therefore have several causes: an impassable cell, an occupied exit, an illegal shoreline crossing, a vanished bridge, an attack position the unit cannot reach, or an order overridden by a script. Check these before assuming the route search failed.

7. Targeting and attacking

1. Obtain a target from a player order, a team script, or automatic acquisition.
2. Check whether it is a valid enemy or a permitted forced target.
3. Choose an eligible weapon for the target and the attacker's current state.
4. Check target category, relevant immunities, range, ammunition, reload and facing requirements.
5. Move into a usable attack position if necessary and possible.
6. Fire when ready. A shot may create a projectile, beam, temporal effect, capture attempt, spawned attacker, or another special action.
7. Resolve the effect using that weapon's associated logic and data.
8. Keep attacking, reacquire, or return to the next order as the target and attacker change.

A field named damage does not always mean ordinary health subtraction. Repair weapons, mind-control weapons and special effects reuse parts of the weapon system. See combat and the linked weapon references on each unit card.

8. Building and the technology tree

1. Start with the owning house and its permitted faction/country.
2. Check technology level, prerequisites, special restrictions and build limit.
3. Find a suitable production facility or construction queue.
4. Spend credits as production advances; insufficient money can interrupt progress.
5. Apply the relevant production conditions, including power and scenario settings.
6. For a completed building, wait for a legal placement. For a completed mobile unit, find a valid exit or delivery mechanism.
7. Create/activate the object, update power and technology availability, and give it its initial order or rally destination.

Prerequisites are not a recommended build order. They are eligibility conditions. Owning a building, having enough power, having money, and having an unblocked factory exit are separate questions. A structure can unlock technology without directly producing that unit.

9. Mission logic is a network of conditions

1. A mission defines named triggers.
2. A trigger belongs to an owner and can be enabled or disabled for a particular difficulty.
3. Events describe what it waits for: elapsed time, a destroyed object, an entered area, an active flag, and so on.
4. A tag associates certain triggers with placed objects or map cells.
5. When the required events have occurred, the trigger runs its action list in order.
6. Actions can show an objective, create a reinforcement, set a flag, enable another trigger, change ownership, or end the battle.
7. Persistence determines whether the attached logic can fire again or is consumed.

Binary-supported detail: the Yuri trigger action runner checks that the trigger is enabled and not destroyed, then walks its action list and calls the action executor for each item. A failure return from one action does not by itself make this a transactional sequence that rolls back earlier actions. The guide's mission ledgers therefore retain action order. They do not present the order of trigger records as a chronological story.

Multiple events can remember that different things happened at different times. They are not necessarily a demand that all visible events happen in the same instant. Enabling a trigger can reset its elapsed-time counters. This is why crossing an invisible boundary can start a new timer or attack wave.

10. Victory is an explicit outcome

1. Read the current objective, including newly displayed objectives.
2. Identify the object, owner, region or flag that represents completion.
3. Satisfy the appropriate mission event, rather than assuming the entire enemy army must always be destroyed.
4. Let any enabling triggers, remaining conditions, delays or scripted sequences complete.
5. The mission invokes its win/lose outcome action and then the score/progression flow.

Destroying a building and capturing it are different events. A scripted capture objective can fail if the building is destroyed. Conversely, destroying an objective may be sufficient even if enemy units remain. Each mission page shows its recorded terminal triggers and the steps feeding them.

11. Saving and loading

1. Pause into the game's options and request Save Game.
2. Initialize the original save dialog and its mission description.
3. In this Android build, append the device-local date and time to that description automatically.
4. Let the original save routine serialize the game's current state into its save format.
5. Acknowledge the original save confirmation through the Android bridge.
6. On load, restore the original game's saved objects and relationships, then continue its simulation.

This is different from saving a screenshot or merely remembering the mission number. A useful save must retain the battle's changing state. The existing QA confirms save/restart/load cases for RA2 and Yuri; it does not establish every possible mission or long-session save condition.

12. How the picture is produced

1. Determine which terrain and objects are visible in the current view.

2. Select the appropriate art, facing, animation, house color and lighting.
3. Draw terrain and objects with the engine's ordering/depth procedures.
4. For voxel models, use their model parts and transforms to produce an image; cached results can avoid repeating work.
5. Combine objects, shadows, effects, selection markers and the interface into the game surface.
6. The host scales/presents that surface on the Android display.

Infantry and many buildings use image sequences; many vehicles and aircraft use voxel models with part transforms. A three-dimensional-looking vehicle does not mean the entire engine is a modern free-camera 3D world. The local voxel research documents the separate model, cache, shadow and battlefield-copy stages. Enhancements described for the desktop presenter must not automatically be assumed enabled in the delivered Android APK.

13. A practical way to study any surprising event

1. Identify the live object and its type card.
2. Select RA2 or Yuri's rules. Compare any version difference on the card.
3. Check that mission's overrides and starting placement.
4. Follow the object's current order and special behavior.
5. Inspect its weapon and the target's armor/immunities.
6. Inspect attached tags, local flags and relevant mission triggers.
7. Separate simulation changes from camera, audio, movie and input effects.
8. Record what you actually observe in a saved test. Treat unobserved possibilities as explanations to test.

For a worked path through an actual scenario, start with Lone Guardian. For changing states within one unit, start with the IFV, Chrono Miner and Slave Miner.

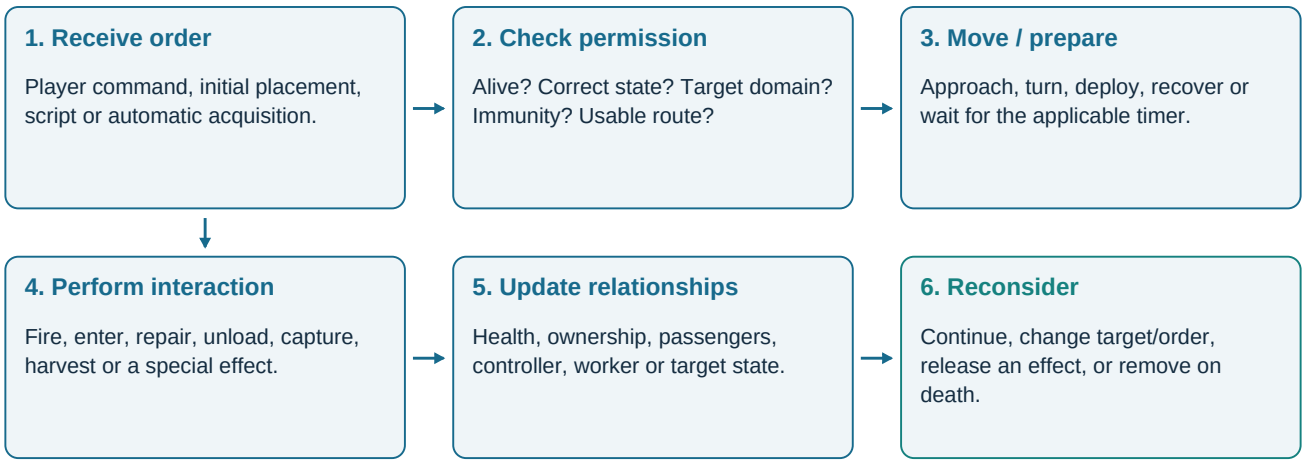
Units: illustrated behavior walkthroughs

Guide index | Engine workflow



A supplied desktop-runtime capture shows mobile objects sharing a battlefield: the selected MCV, tanks and infantry. Upstream project screenshot retained locally; illustrative desktop evidence, not an Android-device test.

One unit: an order becomes an effect



Shared explanatory workflow. Specific unit cards and mission overrides supply the differences.

A shared decision workflow for a mobile object. Explanatory diagram derived from the guide and local data; not a screenshot.

A catalogue entry is not a promise that you can construct it. This book covers every distinct registered type in either effective ruleset, including scenery, hidden/dormant records and weapon-managed objects. The same ID can change role between RA2 and Yuri. Campaign-only additions are listed at the end. Missing properties stay unspecified; a missing permission is not silently treated as yes or no.

Each card gives an ordinary-language workflow, then explicit rule settings. Workflows apply when the object is active and the relevant orders are legal; special map scripts can intervene. Numeric movement/sight/range settings are game units; delays are not automatically wall-clock seconds. For the shared firing procedure see Combat.

Reading an IFV

1. Start empty: the IFV uses its default weapon.
2. Load an infantry passenger.
3. Read that passenger's IFV mode, which chooses the configured vehicle role.
4. Use that mode's vehicle weapon rather than assuming the passenger fires the ordinary hand-held weapon.
5. Unload to return to the default configuration.

A Battle Fortress uses a different open-topped mechanism: eligible passengers contribute their own fire while carried. Neither mechanism makes all transport slots interchangeable.

Selected behavior walkthroughs

This reading edition selects representative mechanisms. The complete companion PDF retains every registered type card, version comparison, weapon path and mission reference.

GI - E1

Registered in: Red Alert 2, Yuri's Revenge.

1. Walk and fire using the standing infantry weapon.
2. Deploy to stop movement and use the deployed attack state; undeploy before moving again.
3. Where permitted, enter an occupiable building and contribute the configured garrison weapon.

Guardian GI - GGI

Registered in: Yuri's Revenge.

1. Fight with the standing weapon while mobile.
2. Deploy to switch into the guarded anti-vehicle/anti-air role.
3. Undeploy to move; the deployed and mobile states have different combat restrictions.

Engineer - ENGINEER

Registered in: Red Alert 2, Yuri's Revenge.

1. Travel to a legal building or repair-hut entry.
2. Enter to capture an eligible enemy/neutral structure, repair a suitable friendly structure, or invoke bridge repair.
3. The mission can react to the entry/capture separately from destruction.

Tanya - TANY

Registered in: Red Alert 2, Yuri's Revenge.

1. Move or swim toward the ordered target.
2. Use her infantry weapon against eligible enemies and C4 against supported close demolition targets.
3. Check the mission ledger: her death is a failure event in several scenarios, independently of the remaining army.

Yuri Clone - YURI

Registered in: Red Alert 2, Yuri's Revenge.

1. Acquire a susceptible target through mind control.
2. Maintain its control relationship rather than dealing ordinary gun damage.
3. Deploy to invoke its close-area psychic attack; immunity and affected-side behavior depend on this version and weapon data.

Chrono Legionnaire - CLEG

Registered in: Red Alert 2, Yuri's Revenge.

1. Teleport to the chosen location and wait out the applicable arrival recovery.
2. Attach its temporal weapon to a legal target and accumulate erasure progress.
3. If interrupted, handle release of the partially erased target; erasure is different from normal shell damage.

Spy - SPY

Registered in: Red Alert 2, Yuri's Revenge.

1. Adopt an allowed enemy-infantry disguise.
2. Avoid detection and reach an eligible enemy building.
3. Apply that building type's infiltration result; the mission can also listen for the infiltration event.

Allied Construction Vehicle - AMCV

Registered in: Red Alert 2, Yuri's Revenge.

1. Move to a clear legal foundation.
2. Deploy into the Allied Construction Yard.

3. Use the resulting building's construction workflow; blocked foundations prevent deployment.

War Miner - HARV

Registered in: Red Alert 2, Yuri's Revenge.

1. Collect resources into its cargo.
2. Drive back to a usable Soviet refinery, dock and unload.
3. Repeat the cycle; use its gun only against supported targets.

Chrono Miner - CMIN

Registered in: Red Alert 2, Yuri's Revenge.

1. Drive outward to a resource field.
2. Collect cargo and invoke its chrono return toward a usable refinery.
3. Unload and repeat; valid destination/placement constraints still apply.

Slave Miner - SMIN

Registered in: Yuri's Revenge.

1. Move next to an accessible deposit and deploy into the linked refinery form.
2. Supply and manage slave workers gathering into that deployed miner.
3. Undeploy to relocate; worker replacement has a configured delay.

IFV - FV

Registered in: Red Alert 2, Yuri's Revenge.

1. With no passenger, use the IFV's default weapon.
2. Load one eligible infantry passenger and read its IFV mode.
3. Switch the vehicle's weapon/turret role accordingly; an Engineer produces a repair role, not an offensive machine gun.

Mirage Tank - MGTK

Registered in: Red Alert 2, Yuri's Revenge.

1. Move visibly to position.
2. When stationary, use its configured disguise behavior.
3. Fire at eligible enemies, briefly exposing the attack state; disguise does not add armor.

Terror Drone - DRON

Registered in: Red Alert 2, Yuri's Revenge.

1. Reach a valid ground victim.
2. Enter its parasite interaction and cause internal damage.
3. Handle victim destruction or supported repair/removal; the drone does not keep acting as an ordinary visible tank.

Magnetron - TELE

Registered in: Yuri's Revenge.

1. Choose the magnetic interaction appropriate to a vehicle or structure.

2. Lift/pull an eligible vehicle, or apply the supported structure attack.
3. Release/place the victim through the special logic; ordinary infantry is not a valid substitute target for its lift.

Gattling Tank - YTNK

Registered in: Yuri's Revenge.

1. Begin firing the appropriate ground/air weapon.
2. Accumulate firing-stage progress and select the corresponding Gattling stage weapons.
3. Lose stage progress when the rules call for cooldown; a short engagement does not start at peak sustained output.

Master Mind - MIND

Registered in: Yuri's Revenge.

1. Acquire eligible enemies into multiple control relationships.
2. Track the number under control.
3. Exceeding safe capacity invokes overload/self-damage behavior; it is not unlimited free capacity.

Floating Disc - DISK

Registered in: Yuri's Revenge.

1. Fly to a combat or drain position.
2. Use its laser against supported targets or attach its drain effect to an eligible structure.
3. Power-drain and refinery-money effects follow the structure-specific drain procedure.

Battle Fortress - BFRT

Registered in: Yuri's Revenge.

1. Load eligible infantry into its passenger slots.
2. Move the heavy transport while allowing supported passenger fire through its open-topped rules.
3. Apply its special crushing permissions when moving over eligible victims/obstacles.

Robot Tank - ROBO

Registered in: Yuri's Revenge.

1. Require the appropriate Robot Control Center and power conditions.
2. Move as a hovering tank across supported land/water and fire at eligible ground targets.
3. Enter the powered-down state when its control conditions fail; mind-control immunity is a separate property.

V3 Launcher - V3

Registered in: Red Alert 2, Yuri's Revenge.

1. Select a valid bombardment target.
2. Raise/launch its spawned rocket under its missile-manager rules.
3. Resolve the rocket's flight and impact separately, then prepare another attack.

Aircraft Carrier - CARRIER

Registered in: Red Alert 2, Yuri's Revenge.

1. Acquire a supported strike target.
2. Launch its managed Hornet aircraft, which fly and attack separately.
3. Recover/rearm or replace attackers according to the spawn-manager settings.

Boomer - BSUB

Registered in: Yuri's Revenge.

1. Move submerged on legal water.
2. Use torpedoes against eligible naval targets or its missile attack against supported land targets.
3. Resolve its independently managed missiles and detection state.

Siege Chopper - SCHP

Registered in: Yuri's Revenge.

1. Fly and use its airborne weapon.
2. Deploy/land into its siege state at a legal position.
3. Use the deployed siege weapon until ordered to return to mobile flight.

Combat, money, power, and special effects

Guide index | Weapon reference | Buildings

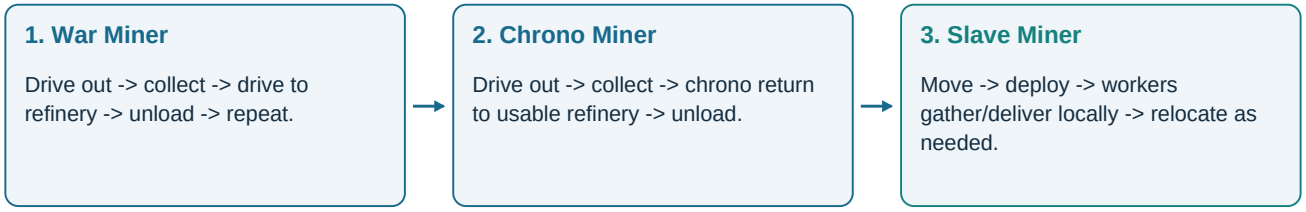
Follow a weapon through its linked records



Ordinary damage is only one path. Mind control, repair and temporal effects need dedicated handling.

The weapon, projectile, warhead and target chain. Explanatory diagram derived from the guide and local data; not a screenshot.

Three different resource loops



Conceptual loops from the rules/manuals. Capacity, timing, destinations and scenario overrides still apply.

The three resource-gathering cycles. Explanatory diagram derived from the guide and local data; not a screenshot.

Evidence: numeric examples below use the extracted effective rules. The per-object cards distinguish RA2 and Yuri and the mission pages list local overrides. Workflows are readable reconstructions of the rules, manuals and identified engine responsibilities; they are not a line-by-line description of every original routine.

1. Follow a shot through four records

1. The attacker chooses an available weapon.
2. The weapon specifies a base effect, firing interval, range and a projectile reference.
3. The projectile specifies how the effect travels and which target domains it can reach.
4. The warhead specifies how the effect interacts with armor and whether special handling is required.
5. The target contributes armor, health, position, ownership and any special protection.
6. The engine resolves the encounter, updates state, and emits animations/sound/events.

The same shell appearance can accompany different damage rules. Conversely, two different-looking attacks can share a warhead. Weapon names are references, not enough information by themselves to predict an outcome.

2. Ordinary damage, explained carefully

A useful starting estimate is: base damage multiplied by the warhead's effectiveness against the target's armor. After that, the real engine can apply veterancy, distance/spread, difficulty, special protection and rounding. This is a learning approximation, not a complete universal damage formula.

The armor-effectiveness list has eleven positions: unarmored infantry; flak-armored infantry; plate-armored infantry; light vehicle; medium vehicle; heavy vehicle; wood structure; steel structure; concrete structure; special category 1; special category 2. These category names are game classifications, not a physical materials model.

1. Find the target's armor class on its card.
2. Find that class's position in the warhead's effectiveness list.
3. Check whether the percentage is zero or strongly reduced.
4. Check whether the weapon/projectile is even allowed to engage that target.
5. Only then compare health, base damage and reload interval.

Zero effectiveness and inability to target are distinct. An attack can also do negligible damage while a different secondary weapon is appropriate. A rapid anti-infantry gun is not automatically a good anti-tank weapon because its animation is busy.

3. Range and reload are independent

1. A target enters weapon range.
2. The attacker turns or positions itself as needed.
3. It waits for its reload and ammunition rules.
4. It releases one shot or a burst.
5. The effect may take more time to reach the target.
6. The next legal firing opportunity is checked again.

The displayed speed stat is not kilometers per hour. The firing interval field is not seconds. A weapon with a burst is not equivalent to multiplying every displayed number together and calling that exact damage per second. Travel time, misses, target changes and special effects matter.

4. Armor, health and experience

1. Health determines how much damage the live object can still sustain.
2. Armor determines which effectiveness multiplier ordinary attacks use.
3. Training eligibility determines whether the object gains rank.
4. Experience and the configured thresholds determine promotion.
5. Veteran/elite ability lists determine which improvements apply.
6. An elite weapon reference can replace an ordinary weapon, changing the attack itself.

Do not assume every elite object receives every possible bonus. The cards retain each explicit ability list. An object's cost, its score value, and the experience contribution associated with destroying it are related data concepts, but are not interchangeable labels.

5. The three resource-gathering workflows

War Miner

1. Locate accessible ore/gems.
2. Drive to the resource and collect into its storage.
3. Return to an eligible refinery when required.

4. Wait for/dock at the unloading point.
5. Convert the delivered resource into the owner's economy and go out again.
6. Use its gun for supported combat situations; that does not make it invulnerable or exempt from needing a refinery.

Chrono Miner

1. Drive outward to reachable resource cells.
2. Gather into its smaller cargo capacity under its rules.
3. Use its chrono return procedure toward an eligible refinery.
4. Complete unloading and resume the outward journey.
5. If the destination cannot be used, the engine must handle that condition; teleport capability does not guarantee a legal refinery always exists.

Slave Miner

1. Move the mobile miner to a usable deposit.
2. Deploy into its linked building form.
3. Its manager supplies slave workers, which travel out, collect and return ore.
4. Process deliveries locally rather than routing a conventional ore truck back to a remote refinery.
5. Replace lost workers according to the configured regeneration delay.
6. Undeploy into the mobile form to relocate when appropriate.

The mobile and deployed forms are separate type records connected by deployment references. They are not two independent income sources simply because both appear in the catalogue.

6. Resource cells and income modifiers

1. A map places resource-bearing cells and controls relevant growth settings.
2. The resource type contributes its configured value and amount.
3. A harvester or worker removes resource while collecting.
4. Delivery converts cargo into usable economic value.
5. Applicable purifier or other modifiers adjust the result.
6. Credits are spent on production, repairs and other supported actions.

Yuri's effective global purifier bonus is 0.25. That is a configured bonus factor, not proof that every map grants it or every resource-related event receives it. A mission can replace rules, provide scripted cash or use a different production setup.

For an Oil Derrick, the effective Yuri record explicitly gives 1,000 startup credits, 20 recurring credits and a recurring delay of 100 simulation frames. Do not read that as 20 credits every 100 seconds. Ownership, capture handling and scenario overrides still matter.

7. Power is an owner-wide service

1. Sum the active output of the owner's generating structures under their current conditions.
2. Sum the requirements of the owner's consuming structures.
3. Compare supply with demand.
4. Apply power-dependent effects: relevant defenses, radar, production conditions, special weapons and powered units.
5. Recalculate when construction, damage, ownership or other state changes require it.

The cards report signed power: positive generates, negative consumes. A missing value is reported as unspecified, not silently changed to zero. A Bio Reactor also depends on its occupants. A Robot Tank depends on its control infrastructure. A Floating Disc can impose a different kind of shutdown through its drain effect.

8. Repairs and selling

1. A building repair request checks whether ordinary repair is permitted.
2. While repairing, spend credits and restore health through the repair procedure.
3. Stop at full health, on cancellation, or when the state no longer permits repair.
4. An Engineer entering a suitable damaged friendly building uses an entry-based repair path instead.
5. A service depot or repair IFV uses its supported vehicle-repair path.
6. Selling, where allowed, removes the owned structure and applies refund/survivor rules rather than applying ordinary weapon damage.

A structure can be capturable but unsellable. A mobile unit can self-heal but not be trainable. These are independent switches, which is why the catalogue reports explicit values rather than inferring all behavior from the object's appearance.

9. Capture, disguise and mind control are different workflows

Engineer capture

1. Select a building that the current Engineer can enter.
2. Check eligibility and ownership.
3. Reach the entry point.
4. Apply capture or the appropriate friendly-repair operation.
5. Notify the mission about the interaction and update the new owner's services.

Spy infiltration

1. Adopt a supported enemy infantry disguise.
2. Move through the battlefield while avoiding units that expose or kill the spy.
3. Enter a spyable enemy structure.
4. Apply the structure-specific infiltration result: power disruption, economic theft, radar reset or a technology/training benefit as supported.
5. Notify any mission-specific spy event.

Ordinary mind control

1. Check the target's eligibility and immunity.
2. Establish a control relationship if the attack succeeds and the controller has the relevant capacity.
3. Let the controlled object act for the controlling side.
4. Maintain or release that relationship according to the controlling mechanism.
5. Handle controller destruction, release and exceptional effects through that mechanism's rules.

Do not describe the Psychic Dominator as an ordinary Yuri Clone with a larger radius. Its persistent capture effect is a separate special weapon behavior. Do not describe a mission's mass ownership-change action as proof that every unit was individually hit by a mind-control weapon.

10. Special combat mechanisms

Mechanism	Readable sequence	Why ordinary damage estimates fail
Chrono erasure	Attach temporal effect; hold target in the temporal interaction; accumulate removal progress; finish or release if interrupted	Health subtraction is not the whole mechanism
Terror Drone parasite	Reach a valid victim; enter its parasite relationship; apply internal damage; handle victim destruction/removal	The visible drone can disappear inside the victim
Giant Squid	Reach a valid naval victim; attach and damage; handle release/interruption	The victim's movement and attachment state matter
Mind control	Validate susceptibility; acquire relationship; maintain capacity and ownership effect	The weapon's damage field can serve special logic
Gattling	Fire and accumulate stage progress; choose stage weapons; lose stage progress when firing ceases	Damage output depends on sustained engagement
Magnetron	Choose vehicle-lift or structure-damage interaction; move the affected vehicle according to magnetic logic; release/place it	Lifting and displacement can be the main outcome
Carrier/Dreadnought/V3	Manage spawned attack objects; launch; resolve their independent travel/attack; reload or replace	The launcher may not deliver the impact directly
Radiation/poison	Create a damaging site/effect; test eligible nearby objects over its lifetime; decay/remove	The consequence lasts beyond the initial attack
C4/Ivan bomb	Reach/place the attached effect; wait for applicable fuse or immediate demolition; detonate or disarm if supported	Detonation is separate from the original targeting click
Transport open-topped fire	Carry infantry; allow eligible passenger weapons to fire through the host's rules	Passengers contribute weapons while the host provides movement

11. Superweapons, one cycle at a time

1. Own the granting structure or receive a scripted grant.
2. Check the weapon's power/availability conditions.
3. Advance its charging state.
4. When ready, accept a legal target or the relevant AI/script request.
5. Execute that weapon's dedicated effect.
6. Reset/recharge, or consume a one-use grant.

Nuclear attack resolves launch/flight/impact and aftermath. Weather Control produces a timed storm. Iron Curtain applies temporary protection to eligible targets. Chronosphere separates a selected group/area from its destination and applies placement consequences. Genetic Mutator transforms eligible infantry through mutation rules. Force Shield protects an area of structures while imposing its configured drawback. These are distinct procedures sharing a charge-and-target interface.

Campaign triggers can set charge, change the next recharge duration, select preferred targets or invoke effects at a waypoint. A campaign countdown can therefore differ from the usual skirmish building's recharge cycle. Read the mission's timer actions before assuming a universal interval.

12. Worked reading exercise

Suppose a tank seems unable to damage a hovering opponent.

1. Look up both type cards and select the right game version.
2. Determine whether the target is treated as airborne at that moment.

3. Follow the attacker's normal and secondary weapon links.
4. Check their projectile target domains.
5. Check warhead effectiveness and the target's immunity fields.
6. Check range, current orders and any local mission override.
7. Only after those checks investigate an input or runtime fault.

This procedure distinguishes a designed limitation, a scenario-specific rule and a possible implementation defect without requiring you to read source code.

Structures: illustrated service walkthroughs

Guide index | Engine workflow



Time Lapse provides a real view of power, research, production and resource infrastructure. Historical local Android-emulator campaign-start capture; not a completed campaign.

A building request has several separate gates

1. Eligibility

Owner/country, technology, prerequisites and build limit.

→

2. Production

Correct queue, credits, power conditions and production progress.

→

3. Placement

A legal footprint, adjacency, terrain and space.

↓

4. Operational service

Produce, unlock, power, reveal, heal, defend, grant a power, or hold occupants.

→

5. State changes

Damage, power loss, capture, infiltration, sale, unloading or destruction.

→

6. Mission reaction

An attached tag can turn an ordinary interaction into objective progress.

A landmark or scenery record does not automatically have an economic or production service.

Separate the eligibility, construction, placement and service stages. Explanatory diagram derived from the guide and local data; not a screenshot.

A catalogue entry is not a promise that you can construct it. This book covers every distinct registered type in either effective ruleset, including scenery, hidden/dormant records and weapon-managed objects. The same ID can change role between RA2 and Yuri. Campaign-only additions are listed at the end. Missing properties stay unspecified; a missing permission is not silently treated as yes or no.

Each card gives an ordinary-language workflow, then explicit rule settings. Workflows apply when the object is active and the relevant orders are legal; special map scripts can intervene. Numeric movement/sight/range settings are game units; delays are not automatically wall-clock seconds. For the shared firing procedure see Combat.

Read these independent questions first

1. Can this owner build it, or must it be found/captured/scripted?
2. Which prerequisites and producer are required?
3. Does it generate or consume power?
4. Does it produce, unlock technology, provide a service, fire a weapon, hold occupants, or only exist as scenery?
5. Can it be captured, infiltrated, repaired or sold?
6. Does this mission attach an objective to this particular instance?

An ordinary occupiable civilian building admits eligible infantry, contributes their garrison fire, and releases them through unload/damage handling. An unoccupied prop has no army-production queue merely because it belongs to the building registry. A light-post record contributes lighting; it is not a power generator unless its power data says so.

Selected behavior walkthroughs

This reading edition selects representative mechanisms. The complete companion PDF retains every registered type card, version comparison, weapon path and mission reference.

Allied Construction Yard - GACNST

Registered in: Red Alert 2, Yuri's Revenge.

1. Receive eligible Allied building requests and advance their construction queue.
2. Require legal placement for a completed structure and add its services when placed.
3. Loss of the yard affects future construction; undeployment depends on the relevant settings.

Power Plant - GAPOWR

Registered in: Red Alert 2, Yuri's Revenge.

1. Join the owner's generating structures when active.
2. Contribute its configured output under current state/damage handling.
3. Recalculate power when captured, damaged, removed or otherwise changed.

Bio Reactor - YAPOWR

Registered in: Yuri's Revenge.

1. Provide Yuri's base power.
2. Accept eligible infantry occupants and increase output through its powered-occupant rules.
3. Release occupants through unloading or destruction handling as supported.

Allied Ore Refinery - GAREFN

Registered in: Red Alert 2, Yuri's Revenge.

1. Provide a resource destination and the configured free-unit delivery on normal construction.
2. Accept compatible miner unloading.
3. Convert delivered cargo into the owner's economy.

Allied War Factory - GAWEAP

Registered in: Red Alert 2, Yuri's Revenge.

1. Produce eligible Allied ground/hover/vehicle-record units.
2. Wait for completion and a usable production exit.
3. Release the unit and its initial order.

Allied Battle Lab - GATECH

Registered in: Red Alert 2, Yuri's Revenge.

1. Satisfy Allied advanced-technology prerequisites while owned.
2. Unlock eligible units/structures as other conditions permit.
3. Removing it can remove eligibility for future construction without erasing every existing advanced unit.

Tech Oil Derrick - CAOILD

Registered in: Red Alert 2, Yuri's Revenge.

1. Accept a legal Engineer capture.
2. Apply the configured capture startup cash and recurring income.
3. On destruction, resolve its explosive death weapon and stop its service.

Civilian Hospital - CATHOSP

Registered in: Red Alert 2, Yuri's Revenge.

1. Accept legal capture and provide the selected ruleset's hospital/healing benefit.
2. Use its infantry-healing settings rather than vehicle repair.
3. Check campaign overrides and eligibility before assuming an entry or global healing method.

Robot Control Center - GAROBO

Registered in: Yuri's Revenge.

1. Provide the required control service for Robot Tanks.
2. Track operational power/control availability.
3. Robot Tanks respond to loss of the service by becoming inactive under their powered-unit rules.

Cloning Vats - NACLON

Registered in: Red Alert 2, Yuri's Revenge.

1. Compare versions: Soviet access in RA2 changes to Yuri technology in the expansion.
2. On eligible infantry production, create a supported duplicate through its cloning procedure.
3. Check cloning restrictions, build limits and the expansion's removal of old recycling behavior.

Grinder - YAGRND

Registered in: Yuri's Revenge.

1. Accept an eligible owned unit for recycling.
2. Remove that unit through the grinding operation.
3. Credit the supported refund/recycling value and release relevant control relationships.

Tesla Coil - TESLA

Registered in: Red Alert 2, Yuri's Revenge.

1. Check operational power and any trooper support.
2. Acquire an eligible target and charge/fire the appropriate Tesla attack.
3. Trooper-assisted behavior uses dedicated rules; it is not simply extra occupants shooting from windows.

Psychic Dominator - YAPPET

Registered in: Yuri's Revenge.

1. Grant its configured special weapon when the owner and operational conditions permit.
2. Advance its charging state, then accept a legal target/request.
3. Execute the dedicated Psychic Dominator attack procedure and reset/recharge as configured.

Psychic Beacon - NAPSYB

Registered in: Red Alert 2, Yuri's Revenge.

1. Exist as a Psychic Beacon object used by mission logic.
2. Receive relevant damage/ownership interactions.
3. Its attached triggers implement the mission-scale control/release consequence; do not infer a universal automatic city effect from its name.

Building a base and reading its dependencies

Guide index | All structures | All units



A working base contains services with distinct dependencies, not interchangeable buildings. Historical local Android-emulator campaign-start capture; not a completed campaign.

The workflows below explain the dependency pattern. They are not a mandatory competitive build order. Country restrictions, game mode, campaign overrides, captured technology, money and placement still determine what is actually legal.

1. Allied development

1. Begin with an Allied Construction Yard, or deploy the supplied Allied MCV on a clear legal foundation.
2. Add power so the base can support its consuming services.
3. Establish resource delivery through an Ore Refinery and Chrono Miner.
4. Add Barracks for eligible infantry and War Factory for eligible vehicle records.
5. Add Air Force Command infrastructure for the aircraft/radar/technology paths it supplies. The American variant has its own record.
6. Add the advanced prerequisites required by the desired unit or structure, such as Battle Lab or Robot Control Center.
7. Use a suitable shoreline for naval production, and preserve valid factory exits and aircraft docking space.
8. Add specialized defenses, information services, income modifiers and superweapons as the rules and battle require.

An Allied Battle Lab unlocks advanced choices; it does not itself eject tanks. The War Factory still performs production. A Robot Control Center supplies a control dependency that continues to matter after the Robot Tank has already been built.

2. Soviet development

1. Start from the Soviet Construction Yard or deploy its MCV.
2. Provide Tesla Reactor power and conventional resource processing.
3. Use War Miners to drive ore to the refinery.
4. Add Barracks and War Factory for the appropriate production queues.
5. Add Radar and advanced research prerequisites as needed.
6. Place the Naval Yard on a legal shore when naval production is useful.

7. Develop high-output power and advanced industry/special weapons where available.
8. Keep the support network alive while attacking; a large army does not replace income or factory access.

Version distinction: RA2's Soviet psychic technology is not simply retained unchanged in Yuri's Revenge. The expansion moves the Yuri-related roster/services and adds other Soviet options. Compare the two columns on the Psychic Sensor/Radar, Yuri infantry and Cloning Vats cards.

3. Yuri development

1. Start with Yuri's Construction Yard or deploy the Yuri MCV.
2. Use Bio Reactors for power; eligible occupants can increase their contribution.
3. Establish Slave Miner processing near a useful deposit.
4. Use its deployed form and managed slaves for the local collection cycle.
5. Add Yuri Barracks and War Factory for the eligible infantry and vehicle records.
6. Add Psychic Radar and Battle Lab prerequisites for the desired technology paths.
7. Use the Naval Yard for Boomers and the corresponding transport options.
8. Add specialized control, recycling, cloning and superweapon facilities according to the battle's needs and explicit restrictions.

The Grinder recycles owned units. Cloning Vats duplicate eligible newly produced infantry. The Bio Reactor uses occupants to support power. These are three different services; a captured soldier can be useful in different ways depending on where you send it.

4. Captured technology and neutral facilities

1. Identify whether the structure permits Engineer capture.
2. Reach it with a surviving compatible Engineer.
3. Transfer ownership through the supported entry operation.
4. Update the services or technology now available to the new owner.
5. Check remaining prerequisites, country restrictions, build limits and production facilities before expecting a new item in the sidebar.

An Oil Derrick provides income. An Airport grants a drop ability. A Machine Shop provides a vehicle-healing benefit. A Secret Lab grants its supported technology choice. None is a generic replacement for every missing prerequisite.

5. Turning a request into a finished building

1. Check whether the building is available to the current owner.
2. Check its prerequisites and limits.
3. Enter it into the correct construction category/queue.
4. Fund progress under the current power and production conditions.
5. When ready, find a legal footprint near permitted base-adjacency sources.
6. Reject blocked cells, unsuitable terrain or other placement violations.
7. Place/activate the structure and recalculate affected services.

Unit production ends differently: the factory needs a usable exit or delivery path. A ready tank is not a building-placement cursor. A naval unit needs an appropriate water exit. An aircraft may require a usable docking allocation.

6. Prerequisite groups in plain language

Reference on a card	Meaning
POWER	A structure from the effective rules' power prerequisite group
FACTORY	A qualifying vehicle-production structure
BARRACKS	A qualifying infantry-production structure
RADAR	A structure in the effective radar prerequisite group
TECH	A qualifying advanced research structure
PROC	A qualifying resource-processing prerequisite, with supported alternatives such as the Slave Miner in Yuri
A named structure ID	That specific type is required by this record
Several comma-separated prerequisites	The record declares several requirements; group alternatives are resolved by their group definitions

The global rules explicitly define the common groups. Capturing one faction's building does not automatically satisfy a different named structure requirement. A mission can add/replace the relevant properties, which is why its override companion matters.

7. Country-name aliases in the reference data

Data name	Player-facing identity
Americans	America
Alliance	Korea
French	France
Germans	Germany
British	Great Britain
Russians	Russia
Confederation	Cuba
Africans	Libya
Arabs	Iraq
YuriCountry	Yuri's faction
Player, BadGuy and other map-created types	Scenario-specific owners/types; read that map's house list and trigger owner references

A trigger's numeric owner selector is not automatically the same index as the list of placed house instances. The ledger reconstructs the country/type list from the baseline list followed by newly introduced map types and shows the matching placed owner where available. The inspected Yuri owner-resolver compares the requested type index with each house's country/type index. Special/unresolved selectors remain labeled instead of assigned to a guessed player.

8. Reading the object cards

Card term	Meaning and limit
Price	Base cost setting; modifiers and mission rules can change the practical amount
Starting health capacity	Full-health setting before applicable modifiers; not a placed object's current health
Technology level	Availability threshold; -1 ordinarily disables normal construction for that type
Required structures/groups	Eligibility conditions, not an optimized build order
Permitted country list	One part of eligibility; it does not override every other restriction
Required/excluded countries	Additional owner restrictions
Build limit	A configured count restriction; unusual negative/special values require their own interpretation
Signed power	Positive output or negative requirement; no explicit field is reported as unspecified
Movement rating	Engine setting, not real-world velocity
Movement zone	Terrain/passability category used with the movement controller
Passenger capacity and size	Different checks: a transport may have slots but reject a unit too large to enter
IFV mode	A passenger's choice of the IFV configuration, not a general weapon number for every vehicle
Veteran/elite improvements	Explicit improvement lists; only applicable improvements should be assumed
Primary/secondary/elite weapon links	Paths into the weapon system; actual selection depends on target and object state
Spyable/capturable/unsellable	Independent interaction permissions
Missing field	The property was not explicitly recovered in that section; the engine may have a default or special case

9. Read a base failure backwards

Suppose no new tank appears after you paid for it.

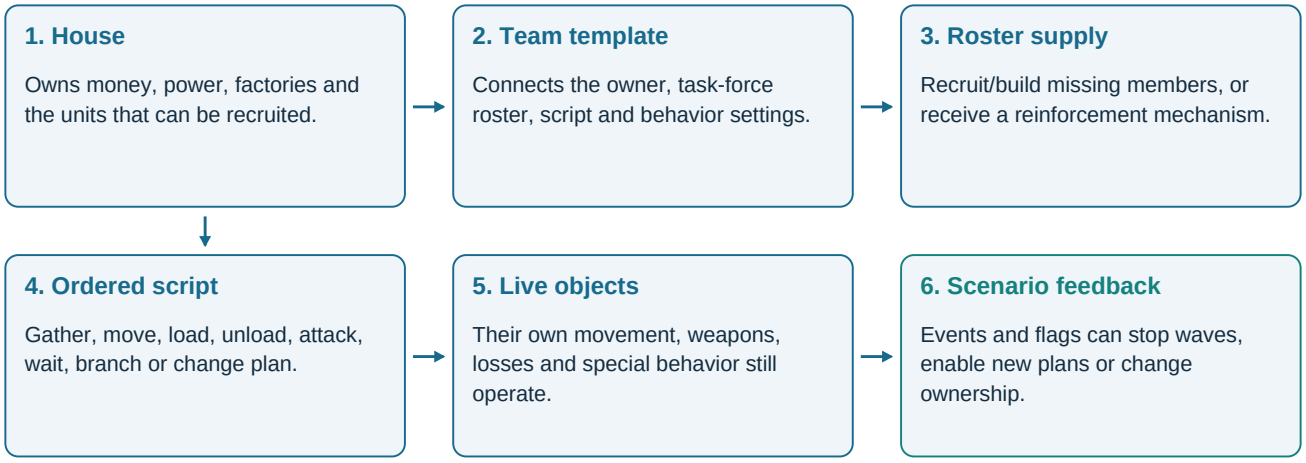
1. Check whether the queue is still progressing or completed.
2. Check current money and the power-dependent production conditions.
3. Check whether the correct factory still exists and belongs to you.
4. Check whether the factory's exit is blocked.
5. Check whether the unit requires a special production/arrival manager.
6. Check mission overrides and scripts that may remove, transfer or interrupt the result.

Suppose the tank exists but does nothing. Now inspect its order, target eligibility, movement restrictions, powered-unit service and any script/input issues. These are different branches of the engine workflow and need different explanations.

How the computer army and mission scripts cooperate

Guide index | Every mission | Global team reference

How an AI force obtains its plan



How owner, roster and plan cooperate to create an attack. Explanatory diagram derived from the guide and local data; not a screenshot.

The opponent is not one all-knowing decision procedure. House-level production, team recruitment, task-force composition, ordered scripts, target acquisition and scenario triggers cooperate. The exact target-ranking internals have not all been traced; the responsibilities and the extracted data below are still useful for understanding the workflow.

1. The vocabulary

Term	Meaning
House	The owner of objects and a pool of money, power and production capabilities
Task force	A required roster, such as several tanks plus infantry
Team type	A template connecting owner, roster, script, recruitment and behavior flags
Team	A live instance trying to fill or execute that template
Script	An ordered list of actions, such as move, unload, attack, wait and repeat
AI trigger	A conditional/weighted opportunity to select a team template
Scenario trigger	A map's event/action rule, often controlling story sequences or waves
Waypoint	A named/numbered location used by scripts and scenario actions

2. Ordinary team production

1. The house must be allowed to produce and have suitable facilities/resources under its rules.
2. Eligible AI decisions consider team templates and their conditions.
3. A selected team asks for its task-force roster.
4. Recruit existing eligible objects or produce missing objects according to the team and house rules.
5. Wait for readiness as required by the template/script.

6. Execute the script one action at a time.
7. Handle destruction, replacement, team limits and completion through the associated procedures.

The global AI reference lists the extracted templates and their rosters. Mission pages list their local templates. A definition's presence does not establish that the team actually spawns on every difficulty, or at all: it must be selected or invoked.

3. Create Team and Reinforcement mean different things

1. A Create Team action instantiates the team logic and attempts to obtain its members through the applicable recruitment/production path.
2. A Reinforcement action supplies new objects through the reinforcement mechanism.
3. A reinforcement-at-waypoint action also identifies the arrival location.
4. A chrono reinforcement uses a different arrival presentation/placement path.
5. The resulting team then follows its script unless other state changes intervene.

This difference explains why one scripted wave can arrive without the visible factory you expected, while another waits for units. It does not prove that all AI objects are free or that no economic rules ever apply.

4. Following a script

1. Start at its current script instruction.
2. Perform that instruction or wait until its completion requirement is satisfied.
3. Move to the next instruction.
4. A branch/repeat instruction changes which instruction is next.
5. A change-script instruction replaces the current plan; a change-team instruction changes the owning team relationship.
6. A destroy/disband instruction may remove objects or remove team organization depending on the actual action.

Script line numbers are reference positions, not seconds. A move can wait for progress, while a sound instruction can complete almost immediately. The script's apparent final action may never be reached if an earlier condition cannot be satisfied.

5. Why a force gathers before attacking

1. Assemble the roster.
2. Move to a staging waypoint or a gather position near a base.
3. Wait for transport loading or other readiness where required.
4. Approach the target or move to a specified enemy-building category.
5. Begin attack/attack-move/hunt under the relevant targeting behavior.
6. Repeat, change plan or dissolve as the script specifies.

Gathering is not necessarily indecision. Likewise, a unit left guarding after its transport unloads may have been intentionally removed from the continuing team. The unload action's continuation setting decides whether the transport, passengers, both or neither continue following that team script.

6. Campaign wave control

1. The map begins with only some triggers enabled.
2. A timer, discovered unit, entered cell or changed flag makes a trigger eligible.
3. Its actions can create a wave and enable its next timer/trigger.
4. Tags can report that a particular building or team-related object has been destroyed.

5. Further actions can disable earlier attacks, change ownership or open the next objective.
6. Difficulty flags can include/exclude parts of this network.

Two missions using the same tank can therefore feel very different. The tank recipe is only one input. Placement, wave timing, team size, script, player economy and mission overrides are separate inputs.

7. Ownership and alliances

1. Each live object belongs to a house.
2. House relationships determine allies/enemies for supported targeting decisions.
3. A mission can change those relationships.
4. It can transfer attached objects, all objects of a particular owner, or a scripted portion of a base through the relevant action.
5. Production, radar/power, target selection and objectives respond to the changed state.

An enemy suddenly becoming friendly can be a story action rather than a weapon hit. The exact trigger is shown in the mission ledger.

8. Difficulty without guessing

1. Read the scenario trigger's Easy/Normal/Hard eligibility flags.
2. Read explicit difficulty sections in the effective rules and any local replacement.
3. Inspect team availability, roster, delays and production settings.
4. Distinguish changed rules from a player taking a different route and activating more triggers.
5. Compare actual runs before claiming an exact difficulty effect that the data alone cannot settle.

The generated mission pages show the raw eligibility flags translated into their named difficulty slots. They do not assume all three are enabled. Rules also contain difficulty-indexed arrays; their ordering is specific to the consuming feature and must not be inferred from the label order on a different record.

9. How to read the detailed ledgers

1. Open the mission from Campaigns.
2. Read its story/objective explanation first.
3. Read terminal gates to see which trigger can declare a winner or loser.
4. Follow incoming enabling, forcing and local-flag dependencies.
5. Inspect attached structures/units/cells to understand what the event applies to.
6. Follow any team link to its roster and ordered script.
7. Check the override table before using a global unit statistic.

The ledger preserves file order only for browsing. Triggers are a graph of possible transitions, not a mandatory sequence that every playthrough takes. Disabled, orphaned and alternative records are still documented; their existence alone is not proof of runtime reachability.

Every campaign, explained as a workflow

Guide index | Mission/AI logic

A mission is a network of guarded transitions



Arrows show dependencies, not a mandatory sequence taken on every playthrough.

Read a mission as a network of guarded transitions. Explanatory diagram derived from the guide and local data; not a screenshot.

Spoilers throughout. The steps below are explanations of each scenario's intended progression and engine mechanisms, cross-checked against its local briefing, campaign table and full trigger ledger. They are not an optimized speedrun route or a claim that each branch has been tested by playing. Map events determine the exact outcome. The linked ledger provides every recovered trigger, attachment, activation link, ordered action, local team plan and explicit rule override.

1. Read the narrative workflow.
2. Open the mission ledger's terminal-gate table.
3. Follow its links to the guard conditions and incoming activators.
4. Follow team links when an action creates a force.
5. Check local overrides before applying ordinary skirmish unit/building rules.

RA2 Allied campaign

Stage	Mission	Detailed engine ledger
1	Operation: Lone Guardian	Open all01t.map
2	Operation: Eagle Dawn	Open all02s.map
3	Operation: Hail to the Chief	Open all03u.map
4	Operation: Last Chance	Open all04u.map
5	Operation: Dark Night	Open all05s.map
6	Operation: Liberty	Open all06u.map
7	Operation: Deep Sea	Open all07t.map
8	Operation: Free Gateway	Open all08u.map
9	Operation: Sun Temple	Open all09t.map
10	Operation: Mirage	Open all10s.map
11	Operation: Fallout	Open all11t.map
12	Operation: Chrono Storm	Open all12s.map

1. Operation: Lone Guardian

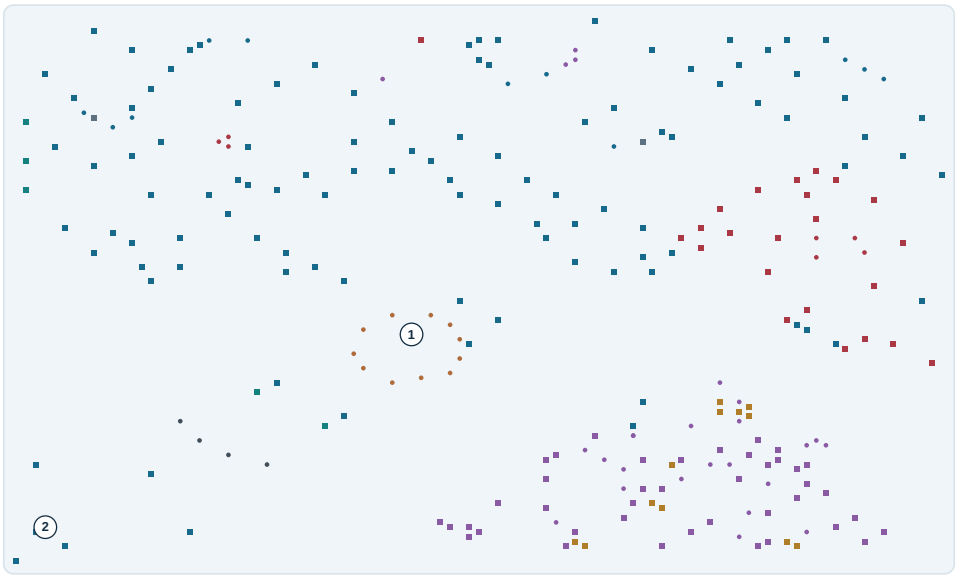
A hero-led introduction that becomes a small base-management mission.

1. Begin with Tanya and the local defenders. The opening ships are a scripted threat to the Statue of Liberty, not merely background decoration.
2. Use Tanya's permitted attacks to remove the Dreadnought threat while preserving her and the protected landmark. Their destruction/death events feed different mission branches.
3. Reach Fort Bradley. The contact/reveal sequence changes which forces and production opportunities you can use.
4. Restore the practical base workflow: power, infantry production and Engineer bridge repair where prompted. These are dependencies for progressing through the terrain, not cosmetic tutorial messages.
5. Destroy the Soviet supply base once the later objective is enabled. The victory sequence is a trigger chain with a delay; Tanya/landmark failure paths remain separate.

Engine lesson: Study how a single contact event can reveal an area, transfer forces, enable a hint and start another timer.

Operation: Lone Guardian

all01t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (117)
- BadGuy1 House (66)
- Other1 House (31)
- BadGuy3 House (12)
- Other3 House (11)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CANEWY04
2. TANY

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/all01t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all01t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

2. Operation: Eagle Dawn

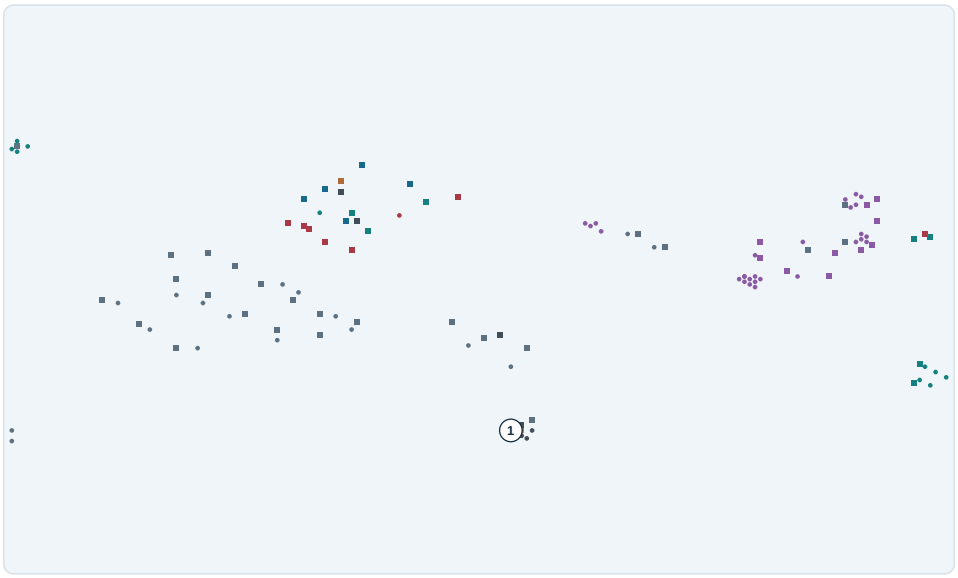
Recover infrastructure before fighting a conventional base battle.

1. Start with Tanya and move through the contact sequence that grants access to scattered friendly forces.
2. Preserve Tanya and use the infantry/specialist tools the mission makes available.
3. Use Engineers for the Soviet-held American facilities, including the Air Force Academy Chapel objective. Capture and destruction are different events.
4. Use the recovered economy and production facilities to build a force suited to the remaining threats.
5. Eliminate the required Soviet forces and let the outcome triggers run; Tanya's loss remains an independent failure.

Engine lesson: Study how ownership transfer changes both the objective state and the player's production capabilities.

Operation: Eagle Dawn

all02s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (42)
- Russians (37)
- Nod (17)
- Americans (9)
- Confederation (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. TANY

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all02s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all02s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

3. Operation: Hail to the Chief

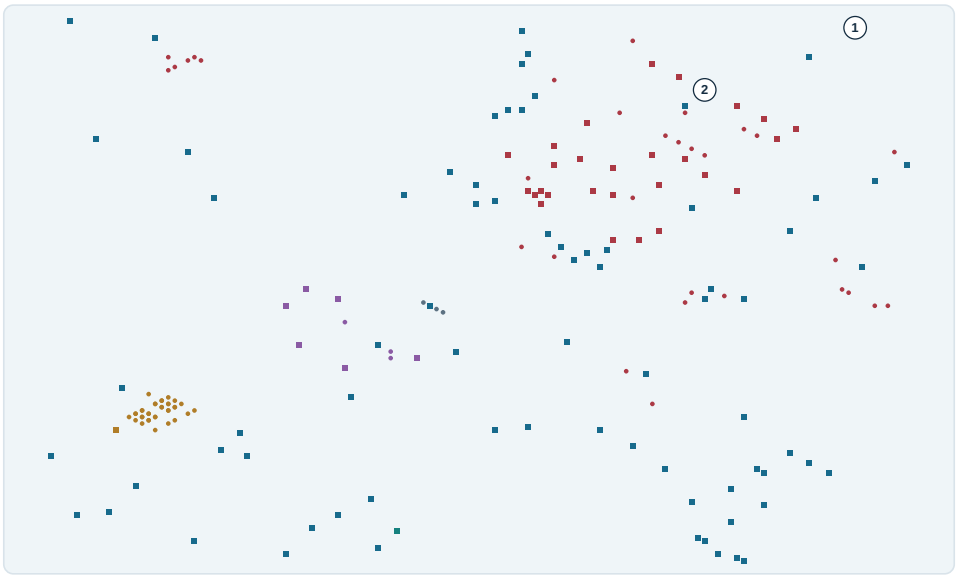
A conventional army must remove a mission-scale psychic-control objective.

1. Initialize the Allied base, economy and local defenders around the mission's starting position.
2. Respond to scripted attacks while producing a force that can reach the White House area.
3. Approach the Psychic Beacon through the available urban terrain; garrison and weapons rules still govern the route.
4. Destroy the tagged beacon rather than assuming that capturing arbitrary enemy buildings completes the story.
5. Allow the scripted release/outcome sequence to resolve after the beacon event.

Engine lesson: Separate an object's ordinary combat role from the scenario action that changes the leaders' control.

Operation: Hail to the Chief

all03u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civilian1 House (75)
- Russian1 House (59)
- Player House (37)
- Russian2 House (9)
- CivilianArmy House (3)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all03u.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. CAWASH01
2. NAPSYB

Initial placement schematic for all03u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

4. Operation: Last Chance

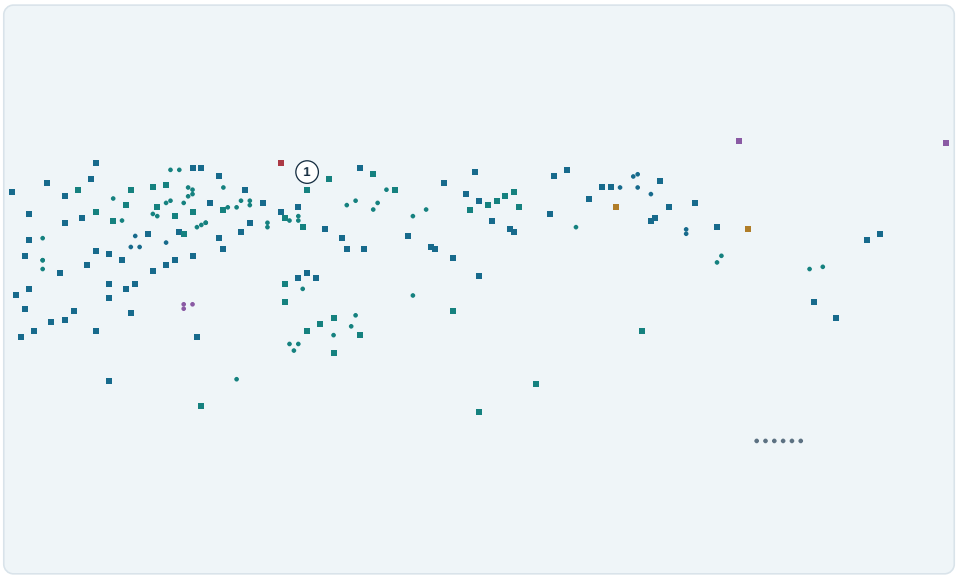
A beachhead, economy and objective attack operate under a deadline.

1. Complete the shore landing and establish a usable base.
2. Begin harvesting and production quickly enough to support the coming attack.
3. Track the Psychic Amplifier countdown and its difficulty/activation triggers. The timer is a mission mechanism, not the recharge indicator of every enemy structure.
4. Reach and destroy the Chicago amplifier before its active failure condition is satisfied.
5. Let the destruction/success chain finish; expiration follows a separate branch even if much of the enemy army is already gone.

Engine lesson: Read the timer set/start actions and compare them with the destruction gate.

Operation: Last Chance

all04u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civilian1 House (96)
- Russian1 House (89)
- Player House (6)
- Russian3 House (5)
- Russian2 House (2)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. NAPSYA

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all04u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all04u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

5. Operation: Dark Night

Infiltration and demolition use different specialists and event types.

1. Keep Tanya alive while scouting the route to the Soviet research facilities.
2. Use the mission's infiltration tools to enter the Soviet Battle Lab and reveal/progress the objective chain.
3. Use disguise, power disruption and direct attacks according to each unit's supported interactions; a Spy is not an Engineer capture unit.
4. Neutralize both nuclear silos required by the objective. Removing just one does not satisfy the pair's completion logic.
5. Preserve Tanya until the mission issues its outcome after the silo conditions.

Engine lesson: Compare infiltration, reveal actions and multiple-object destruction conditions.

Operation: Dark Night

all05s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- RedTeam House (131)
- Civilians House (41)
- OrangeTeam House (36)
- YellowTeam House (35)
- Mr. Neutral House (18)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. NAMISL
2. TANY

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all05s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all05s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

6. Operation: Liberty

Recover the capital through a sustained production battle.

1. Connect with the Allied Pentagon forces through the mission's initial command-transfer sequence.
2. Keep the recovered base functional with income, power and accessible production exits.
3. Defend against the scripted Soviet attacks while assembling suitable ground and support forces.
4. Push into the occupied city and remove the required Soviet force/base ownership scopes.
5. Wait for the all-required-enemies completion gate and score sequence.

Engine lesson: Study the difference between a new unit being reinforced and an existing unit changing owner.

Operation: Liberty

all06u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (97)
- BadGuy1 House (68)
- BadGuy2 House (39)
- Player House (23)
- House Elania (1)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CAWASH01
2. CAWA2A

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all06u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all06u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

7. Operation: Deep Sea

Naval control, land infrastructure and scripted reinforcements interact.

1. Organize the Hawaiian position and its land-based economy.
2. Use appropriate naval units for surface ships, submarines and aircraft; one naval weapon does not cover every target domain.
3. Respond to the arriving Soviet fleet and island-related wave triggers.
4. Remove the required Soviet sea and land presence using suitable bombardment and landing tools.
5. Satisfy the mission's destruction gates rather than assuming that sinking a single named ship ends the battle.

Engine lesson: Follow naval target eligibility and the separate team arrival scripts.

Operation: Deep Sea

all07t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie2 House (108)
- Other1 House (45)
- Player House (39)
- Civie1 House (7)
- Civie3 House (7)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all07t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all07t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

8. Operation: Free Gateway

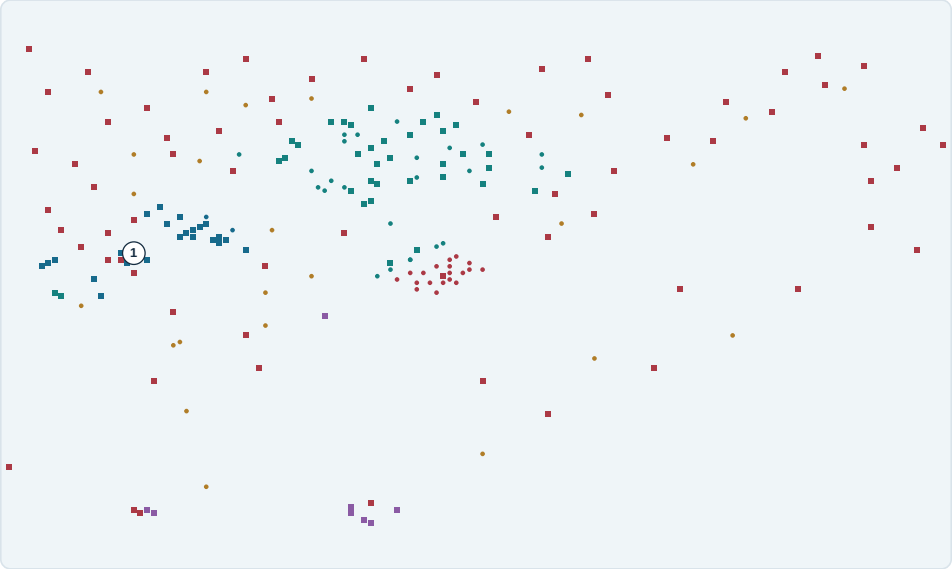
A hero strike releases a city before the remaining battle is finished.

1. Keep Tanya safe while advancing through the St. Louis opening.
2. Find the Soviet Psychic Beacon and plan an attack using legal targets/routes.
3. Destroy the beacon to invoke the mission's control-release and progression actions.
4. Use the resulting position and available army to remove the remaining required Soviet forces.
5. Keep Tanya alive through the actual victory gate; beacon destruction and complete mission success are separate stages.

Engine lesson: Read how an objective event changes ownership and enables the next objective.

Operation: Free Gateway

all08u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (88)
- BadGuy1 House (60)
- BadGuy2 House (27)
- Evil House (25)
- House Elania (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. NAPSYB

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all08u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all08u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

9. Operation: Sun Temple

Deny stolen technology and then remove its defenders.

1. Use the supplied strike force to approach the Yucatan research area.
2. Identify the Soviet attempts to reproduce Prism technology.
3. Capture or destroy the relevant technology targets as the objective permits. This is one of the cases where the briefing explicitly allows more than one kind of result.
4. Eliminate the base defending the research site with the facilities and specialists made available.
5. Complete both the technology-denial and base-destruction parts of the scenario.

Engine lesson: Compare alternative permitted objective outcomes with a mission that requires capture only.

Operation: Sun Temple

all09t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- BadGuy1 House (144)
- Civie1 House (40)
- BadGuy2 House (15)
- Other2 House (12)
- House Balabos (9)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all09t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all09t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

10. Operation: Mirage

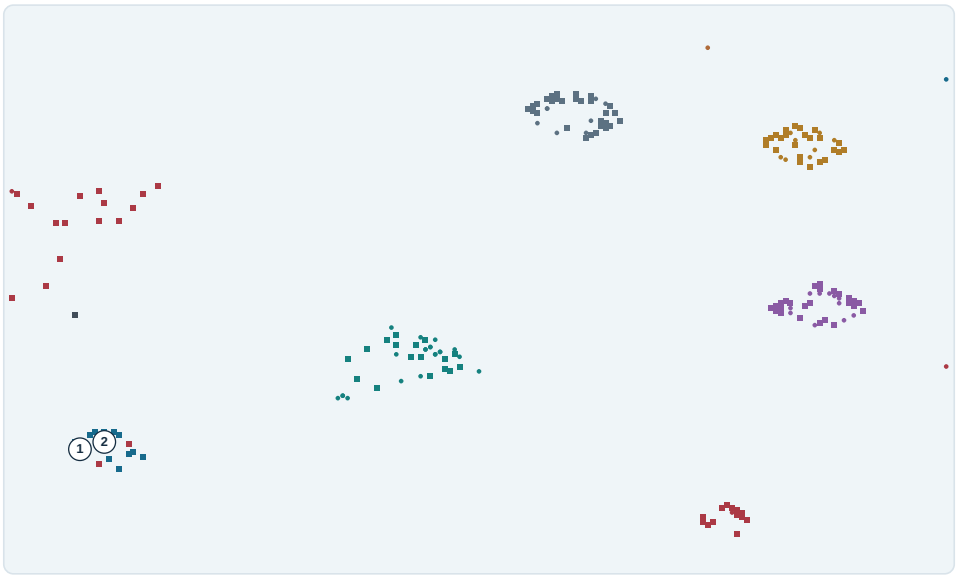
Protect a critical laboratory while destroying the attacking army.

1. Identify Einstein's laboratory as the protected object and organize defenses around usable approaches.
2. Keep economy, power and production functioning during the enemy's attack sequences.
3. Use the available Allied technology to counter the mix of ground and air threats.
4. Move from defense to eliminating the required Soviet forces without exposing the laboratory.
5. Reach the victory conditions while avoiding the lab-destruction failure branch.

Engine lesson: A defend objective remains relevant during your offensive; it is not automatically cleared by building a large army.

Operation: Mirage

all10s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Player House (38)
- German1 House (37)
- German3 House (36)
- Civilian House (32)
- German2 House (32)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. GACSPH
2. CALAB

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all10s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all10s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

11. Operation: Fallout

A construction milestone is followed by neutralizing a nuclear threat.

1. Establish a sustainable base near Soviet-controlled Cuba.
2. Meet the relevant prerequisites and construct the Chronosphere objective.
3. Watch the mission actions that respond to the building milestone and nuclear threat.
4. Use the available transfer, transport and assault tools to neutralize the specified Soviet missile threat on Cuba.
5. Satisfy the objective flags and outcome gate; a completed Chronosphere alone is not the whole mission.

Engine lesson: Compare a build-type event with later destruction-based objective gates.

Operation: Fallout

all11t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- BadGuy2 House (80)
- Civie1 House (77)
- Player House (40)
- House Elania (18)
- BadGuy3 House (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. NAMISL

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all11t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all11t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

12. Operation: Chrono Storm

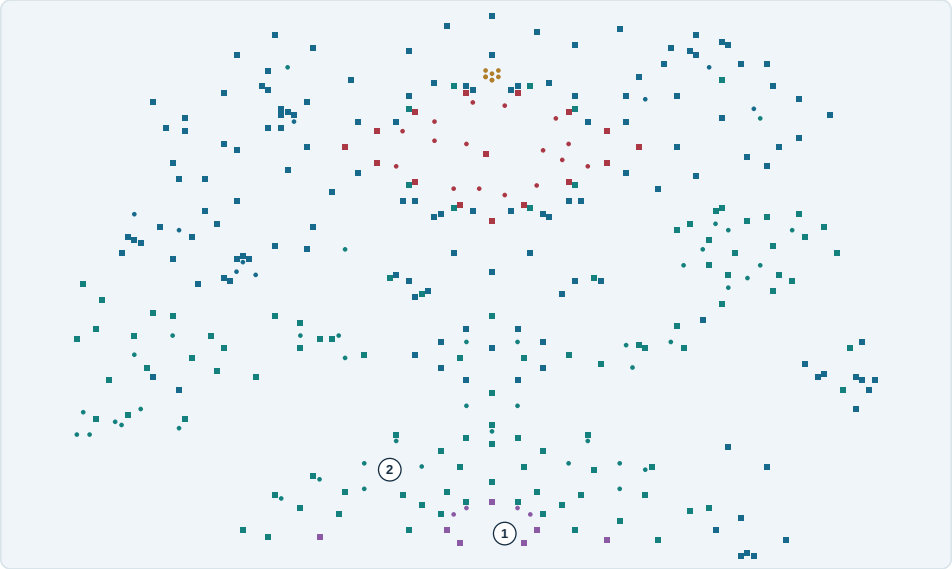
A staged invasion uses reinforcement and defended-area conditions.

1. Secure the initial Moscow arrival area with the supplied strike force.
2. Clear the required arrival zone so the chrono reinforcement sequence can proceed.
3. Use the reinforced army to penetrate the capital and approach the Kremlin area.
4. Eliminate the designated Elite Black Guard defenses rather than interpreting the briefing as a requirement to destroy every city prop.
5. Let Tanya's Romanov capture/finale sequence and the mission outcome actions complete.

Engine lesson: The ending combines combat conditions and a scripted story sequence; it is not simply a tank-damage calculation on Romanov.

Operation: Chrono Storm

all12s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (150)
- BadGuy1 House (146)
- BadGuy2 House (32)
- BadGuy3 House (11)
- Other1 House (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CARUS03
2. NAMISL

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/all12s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all12s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

RA2 Soviet campaign

Stage	Mission	Detailed engine ledger
1	Operation: Red Dawn	Open sov01t.map
2	Operation: Hostile Shore	Open sov02t.map
3	Operation: Big Apple	Open sov03u.map
4	Operation: Home Front	Open sov04s.map
5	Operation: City of Lights	Open sov05u.map
6	Operation: Sub-Divide	Open sov06t.map
7	Operation: Chrono Defense	Open sov07s.map
8	Operation: Desecration	Open sov08u.map
9	Operation: The Fox and the Hound	Open sov09u.map
10	Operation: Weathered Alliance	Open sov10t.map
11	Operation: Red Revolution	Open sov11s.map
12	Operation: Polar Storm	Open sov12s.map

1. Operation: Red Dawn

A scripted invasion teaches basic force control and urban fighting.

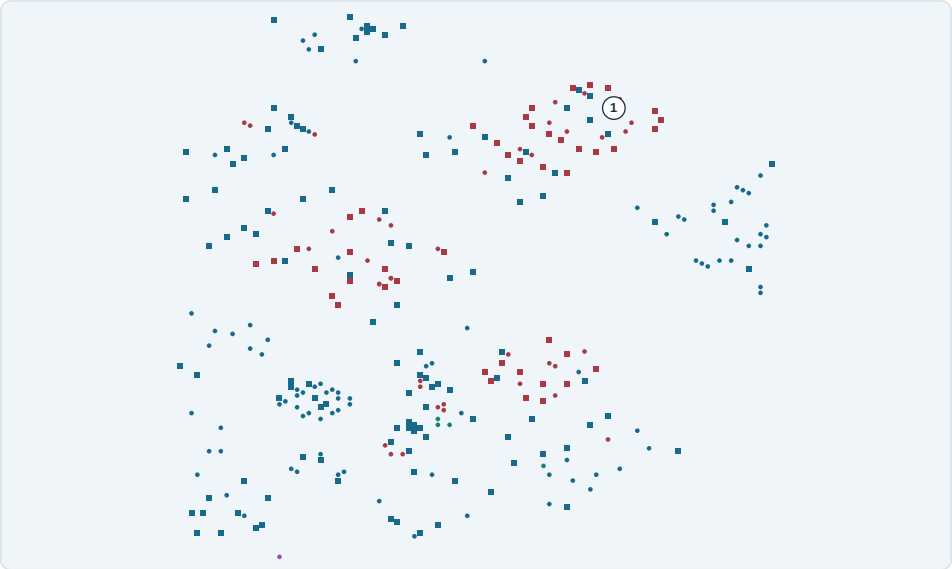
1. Take command of the Soviet landing/invasion force and follow the initial contact sequences.
2. Use the provided production and reinforcement opportunities to maintain the attack.
3. Advance through Washington while using garrison, anti-infantry and demolition tools appropriately.
4. Destroy the required Pentagon objective parts/linked scope.

5. Allow the objective-destruction chain to declare the mission outcome.

Engine lesson: A visually large landmark can consist of multiple placed type records and associated tags.

Operation: Red Dawn

sov01t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (209)
- BadGuy1 House (85)
- BadGuy2 House (4)
- Player House (1)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov01t.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. CAWA2A

Initial placement schematic for sov01t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

2. Operation: Hostile Shore

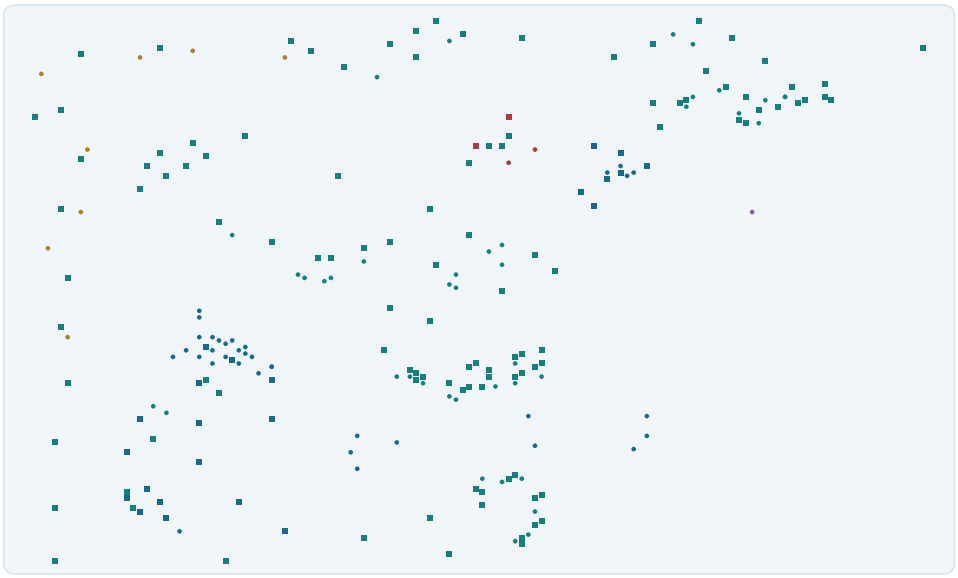
An amphibious landing develops into an annihilation objective.

1. Use the supplied naval support to establish a safe Florida landing.
2. Deploy a base on a legal site and begin the normal resource/production cycle.
3. Use Vladimir's support according to its scripted role; it is separate from your own factory queue.
4. Remove enemy positions and naval threats with appropriate land/sea weapon domains.
5. Eliminate the required remaining enemy forces and satisfy the outcome trigger.

Engine lesson: Distinguish autonomous/scripted allied support from directly owned player units.

Operation: Hostile Shore

sov02t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (153)
- Americans (58)
- Nod (8)
- Germans (4)
- Russians (1)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov02t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov02t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

3. Operation: Big Apple

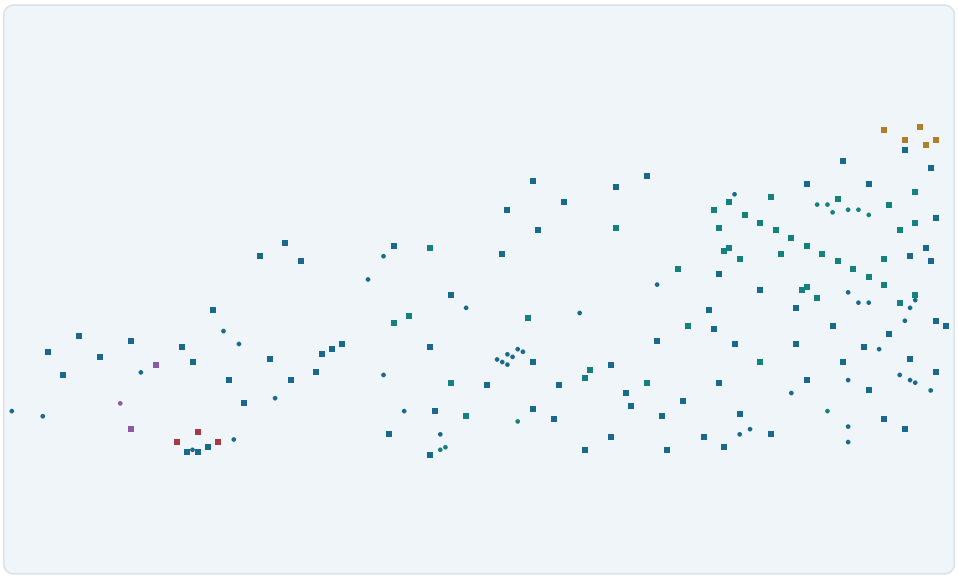
Capture enables construction, then defense enables mission completion.

1. Reach the Allied Battle Lab in New York with a viable capture force.
2. Enter it with an Engineer rather than destroying it; this objective depends on captured technology.
3. Use the newly enabled Psychic Beacon construction opportunity.
4. Build and defend the beacon through the mission's attack/countdown sequence.
5. Keep the objective alive until the completion/control sequence resolves.

Engine lesson: This is a clear state machine: acquire technology, build objective, defend while a timer/flags progress, complete.

Operation: Big Apple

sov03u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (123)
- BadGuy1 House (52)
- Other3 House (5)
- Other2 House (3)
- Player House (3)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov03u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov03u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

4. Operation: Home Front

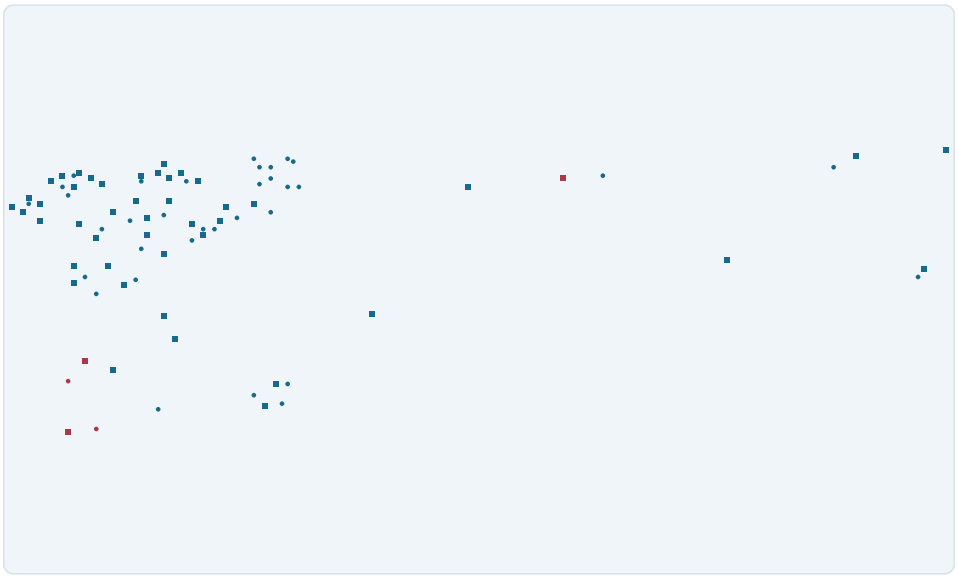
Defend the home territory, then remove the invading force.

1. Develop the Soviet base and its economy.
2. Prepare ground and air defenses against the Korean invasion scripts.
3. Preserve operational power and production as waves arrive.
4. Counterattack the invading enemy positions and remaining forces.
5. Satisfy the mission's required enemy-destruction gates.

Engine lesson: Country identity affects available aircraft, while mission scripts determine when attacks arrive.

Operation: Home Front

sov04s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civilian House (79)
- Player House (5)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov04s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov04s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

5. Operation: City of Lights

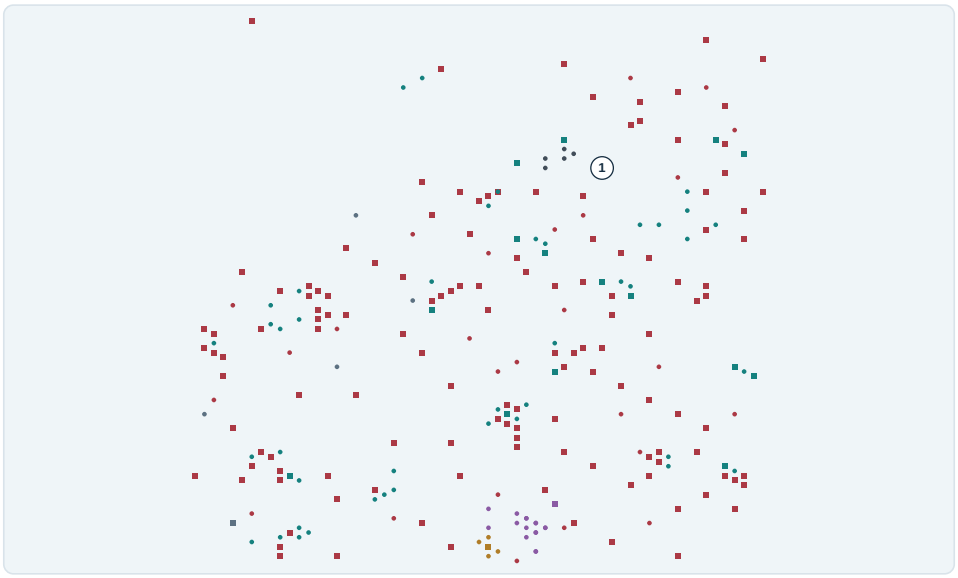
A landmark becomes a scenario-specific weapon through support conditions.

1. Advance the supplied Soviet infantry through Paris.
2. Bring the required Tesla Troopers into the Paris Tower's supported charging interaction.
3. Energize the tower with the required trooper support; it is a special map setup, not an ordinary buildable tower queue.
4. Defend the tower and its support while its enabled weapon attacks the Allied opposition.
5. Remove the remaining required Allied troops without losing the protected objective.

Engine lesson: Follow the local Paris Tower rules and tags instead of copying the global Tesla Coil description blindly.

Operation: City of Lights

sov05u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (157)
- BadGuy1 House (57)
- Player House (17)
- Other3 House (5)
- Other1 House (5)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov05u.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. CAPARS01

Initial placement schematic for sov05u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

6. Operation: Sub-Divide

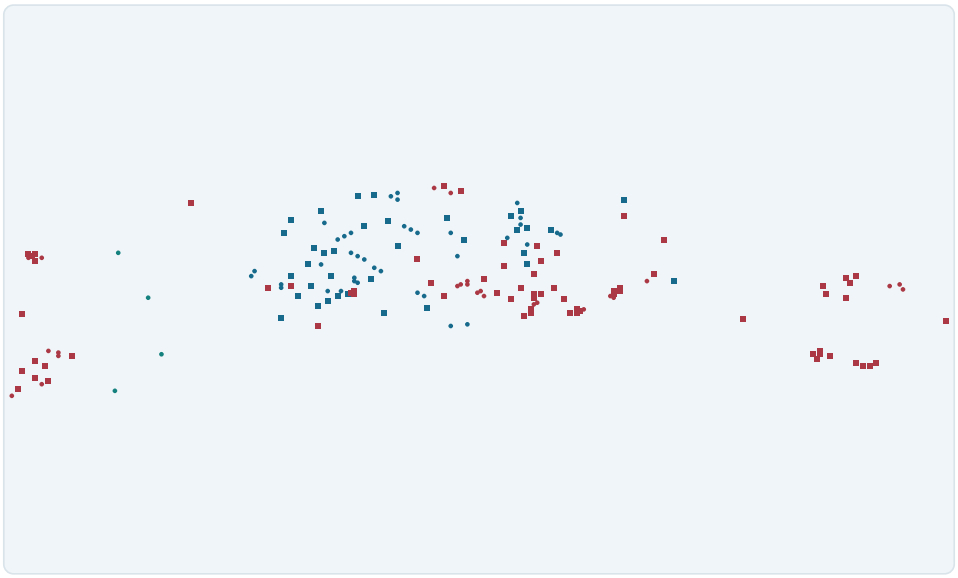
Island economy supports a naval destruction objective.

1. Establish a working base in the Hawaiian island position.
2. Develop income and naval production with usable shoreline exits.
3. Respond to American defenses and the Korean fleet-related reinforcement scripts.
4. Use submarine, anti-air and bombardment roles against the targets each can legally engage.
5. Destroy the required Allied Navy ownership scope to reach the outcome chain.

Engine lesson: The mission explicitly emphasizes naval survival/destruction, which can differ from a generic all-buildings objective.

Operation: Sub-Divide

sov06t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral House (95)
- BadGuy1 House (73)
- Other1 House (4)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov06t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov06t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

7. Operation: Chrono Defense

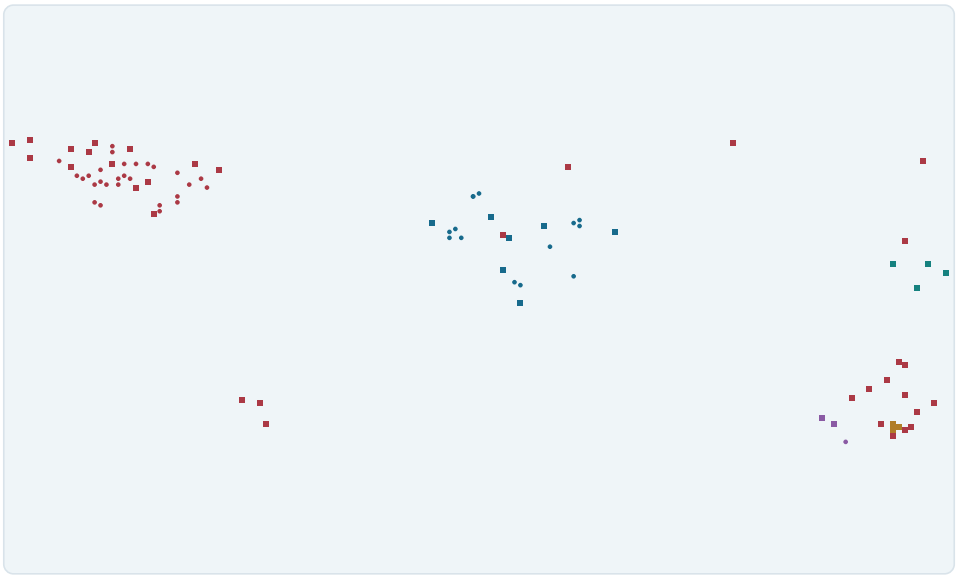
A protected Battle Lab must survive successive chrono attacks.

1. Identify the Soviet Battle Lab and organize defenses around its possible attack directions.
2. Develop economy and production while preserving that lab.
3. Respond to chrono reinforcement waves, which need not travel visibly from the edge of the map.
4. Track the scenario's staged timers and flags; a quiet interval can be a preparation period.
5. Keep the lab alive through the required defense sequence and its final completion condition.

Engine lesson: Study repeated timer resets and reinforcements that arrive at internal waypoints.

Operation: Chrono Defense

sov07s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (62)
- Player House (21)
- Other3 House (4)
- Other1 House (3)
- Other2 House (3)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov07s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov07s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

8. Operation: Desecration

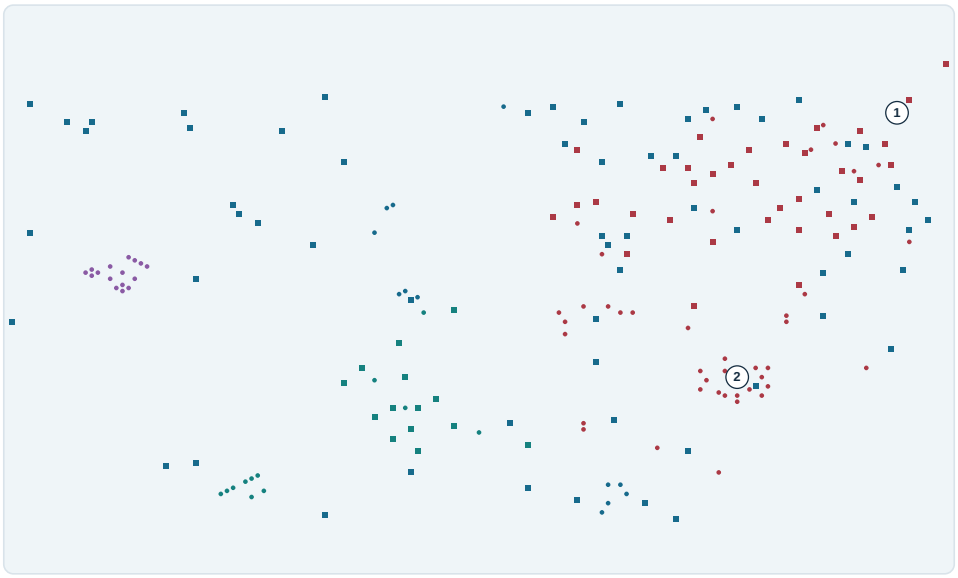
The target must change owner, so demolition is the wrong final interaction.

1. Establish your forces in the Washington battle against Vladimir's army.
2. Maintain economy, power and an escort capable of clearing a route.
3. Approach the White House while preserving the target structure for capture.
4. Send a surviving Engineer through the legal entry to take control of the White House.
5. Let the capture/ownership event advance the victory sequence.

Engine lesson: A capture objective uses a different terminal gate from a destroy-the-headquarters objective.

Operation: Desecration

sov08u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Vlad2 House (78)
- Civilian1 House (76)
- Vlad1 House (26)
- Player House (16)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CAWASH01
2. NAPSYB

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov08u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov08u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

9. Operation: The Fox and the Hound

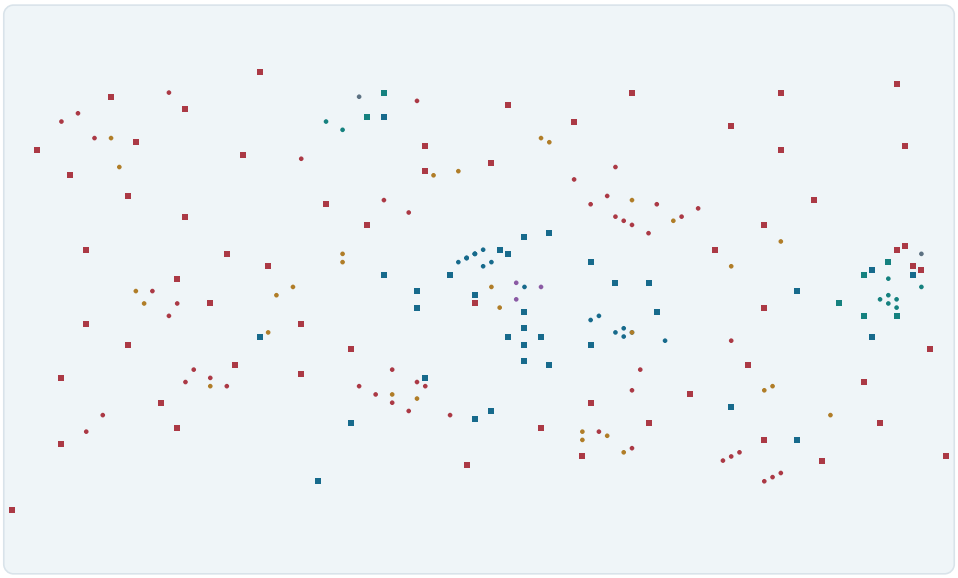
A small force exploits disguise and mind control rather than base production.

1. Begin with the mission's supplied psychic/infiltration tools near San Antonio.
2. Use susceptible enemy soldiers as a resource through supported mind control.
3. Use infiltrators and power disruption where useful to open access around the Alamo.
4. Expose and acquire the President through mind control, keeping the target alive for that interaction.
5. Complete the President-control event; destroying him is not a substitute for capturing him.

Engine lesson: Ownership effects can be the required result even when almost no ordinary production is available.

Operation: The Fox and the Hound

sov09u.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (107)
- Americans (52)
- Germans (30)
- Alliance (16)
- GDI (3)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov09u.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov09u.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

10. Operation: Weathered Alliance

Information acquisition reveals the real objective.

1. Build up the Soviet position in the Virgin Islands.
2. Reach the Allied Battle Lab with an Engineer while protecting the capture target.
3. Capture the lab to locate/reveal the Weather Control Device through the scenario actions.
4. Reach and destroy that device with suitable ground/naval/transport tools.
5. Complete the target-destruction gate after the information stage.

Engine lesson: A map reveal is an action caused by an objective, not proof that the target was created at that instant.

Operation: Weathered Alliance

sov10t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- BadGuy3 House (94)
- BadGuy1 House (64)
- BadGuy2 House (55)
- Civie1 House (42)
- Player House (5)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov10t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov10t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

11. Operation: Red Revolution

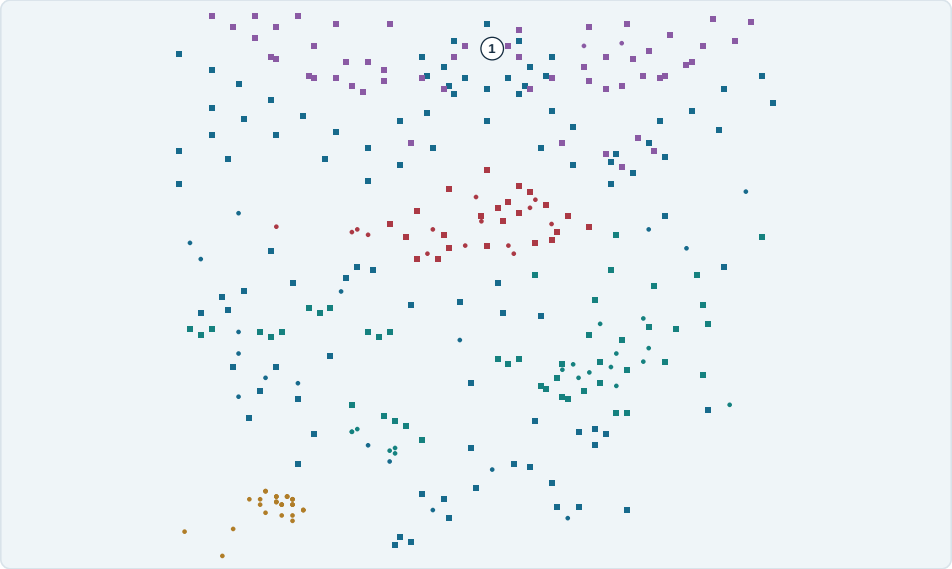
An offensive must survive scripted pressure on its economy and base.

1. Prepare the Soviet base for the Moscow battle against Yuri's forces.
2. Keep income and power available during the scenario's timed and reactive attacks.
3. Build a force that can cope with psychic-control threats and defended approaches.
4. Penetrate the Kremlin area and destroy the designated Yuri headquarters scope.
5. Allow the headquarters-destruction chain to declare success.

Engine lesson: Separate baseline unit immunity/counterplay from mission-specific waves and available resources.

Operation: Red Revolution

sov11s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civie1 House (116)
- BadGuy1 House (65)
- BadGuy3 House (57)
- BadGuy2 House (37)
- Player House (34)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov11s.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. CARUS03

Initial placement schematic for sov11s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

12. Operation: Polar Storm

A final base assault opposes an enemy transfer network.

1. Establish the Soviet position near Point Hope, Alaska, with the limited starting force.
2. Create a reliable economy and defenses against the Allied attacks.
3. Use the available high-technology assault options to reach the defended Chronosphere.
4. Destroy the designated Chronosphere objective before the enemy's pressure defeats you.
5. Resolve the objective event and campaign ending; unrelated scenery is not an extra victory requirement.

Engine lesson: Enemy chrono arrivals are scenario-controlled attack delivery, while destroying the objective ends the strategic threat.

Operation: Polar Storm

sov12s.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- BadGuy3 House (135)
- BadGuy1 House (71)
- BadGuy2 House (18)
- Civie1 House (8)
- Player House (4)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. GACSPH

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov12s.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov12s.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

Yuri's Revenge Allied campaign

Stage	Mission	Detailed engine ledger
1	Operation: Time Lapse	Open all01umd.map
2	Operation: Hollywood and Vain	Open all02umd.map
3	Operation: Power Play	Open all03umd.map
4	Operation: Tomb Raided	Open all04dmd.map
5	Operation: Clones Down Under	Open all05umd.map
6	Operation: Trick or Treaty	Open all06umd.map
7	Operation: Brain Dead	Open all07smd.map

1. Operation: Time Lapse

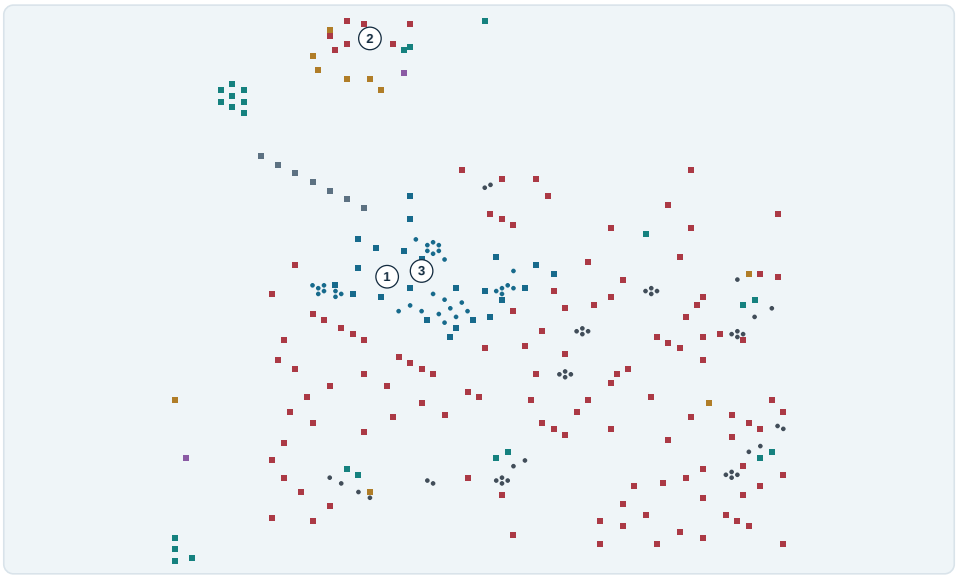
A mission-specific power network drives a time-travel state change.

1. Defend the Time Machine and organize the starting San Francisco forces.
2. Capture the marked power sources so the mission's power-objective conditions become satisfied.
3. Preserve the machine while the scripted travel sequence changes the scenario state.
4. Use the post-travel position and available forces to reach Alcatraz.
5. Destroy the Psychic Dominator and complete the mission outcome while respecting machine/failure conditions.

Engine lesson: The map defines individual power-plant copies and counting flags. This is more specific than simply having any four ordinary Power Plants.

Operation: Time Lapse

all01umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (120)
- Americans (58)
- YuriCountry (41)
- PowerHouse (24)
- Arabs (10)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CALAB
2. YAPPET
3. CATIME01

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all01umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all01umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

2. Operation: Hollywood and Vain

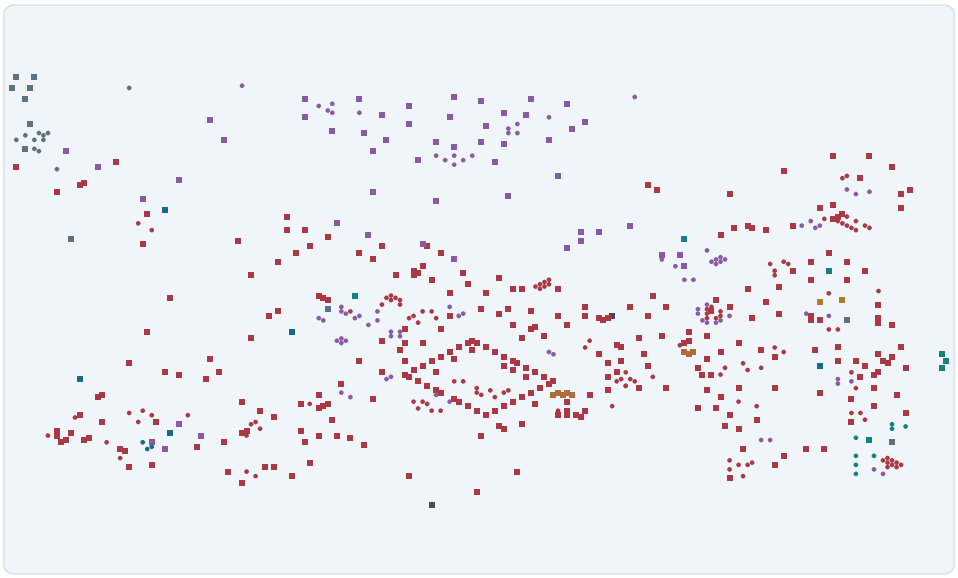
Disable an economic network before finishing the enemy base.

1. Establish the Allied force in Los Angeles and defend its production/economy.
2. Find the individual Grinder objectives and the civilian/hero-related contact opportunities.
3. Destroy all Grinders counted by the mission flags. A visible cash-generating story process is represented by scripts and object relationships.
4. Destroy the remaining required Yuri buildings.
5. Let the combined Grinder/base completion gates declare victory.

Engine lesson: The patch archive supplies the effective version of this map. Individual Grinder flags combine into the larger objective.

Operation: Hollywood and Vain

all02umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (401)
- YuriCountry (127)
- Yuri2 (22)
- Nod (10)
- Russians (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: expandmd01.mix/all02umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all02umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

3. Operation: Power Play

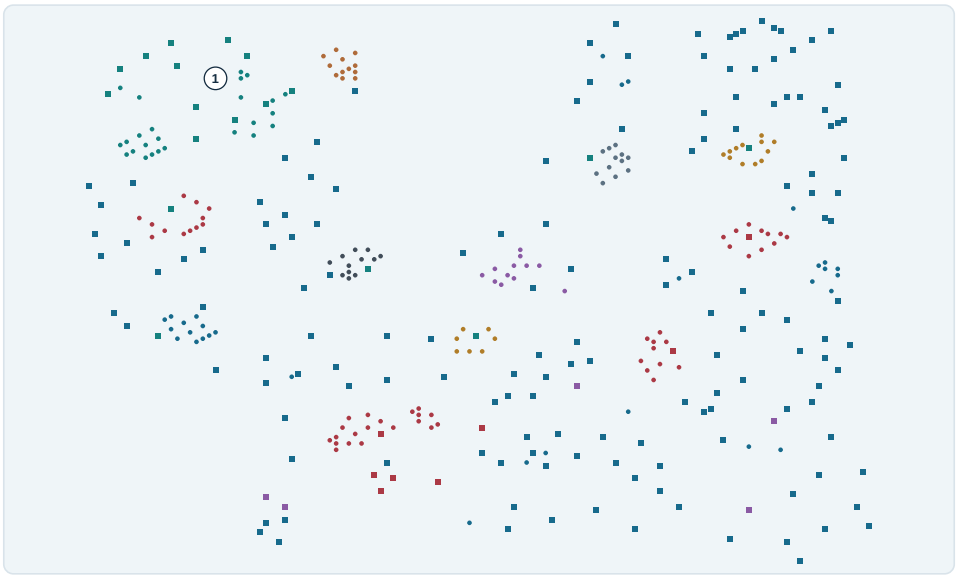
A nuclear threat and a liberation objective are separate stages.

1. Develop the Allied position in Seattle while monitoring the silo-related time pressure.
2. Produce a force suited to reaching Yuri's Nuclear Silo.
3. Destroy the specified silo to remove the immediate threat.
4. Liberate the MassiveSoft campus through the mission's required object/ownership conditions.
5. Complete both parts of the objective network rather than stopping at the silo explosion.

Engine lesson: A story company/campus label is implemented through placed objects and trigger scopes.

Operation: Power Play

all03umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (170)
- Americans (49)
- YuriCountry (32)
- Russians (12)
- Nod (12)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. NAMISL

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all03umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all03umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

4. Operation: Tomb Raided

Removing power structures releases a captive and changes mission capability.

1. Approach the Egyptian Great Pyramid objective and identify its associated Bio Reactors.
2. Destroy the required reactors around the prison/research location.
3. Allow the Einstein rescue sequence and any resulting special capability to occur.
4. Use the available force and enabled tools against Yuri's base.
5. Destroy the required base scope while avoiding the scenario's protected-unit failure branches.

Engine lesson: Rescue can be implemented as an enabled sequence following several object-destruction flags.

Operation: Tomb Raided

all04dmd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- YuriCountry (63)
- Neutral (39)
- Arabs (24)
- Americans (11)
- French (9)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. YAPPET

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all04dmd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all04dmd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

5. Operation: Clones Down Under

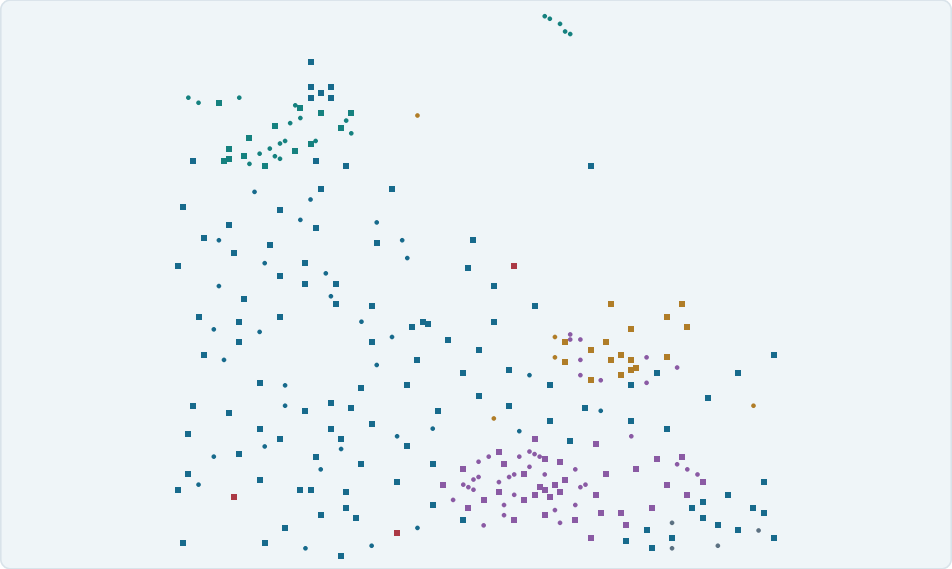
A technology-production facility is the first objective, not the entire battle.

1. Establish the Allied Sydney position and a functioning economy.
2. Scout for Yuri's cloning facilities while responding to the local attack teams.
3. Destroy the specified Cloning Vats to interrupt that objective branch.
4. Remove the remaining required Yuri forces with appropriate counters to their control/air/armor roles.
5. Complete the remaining-force and cloning-objective gates.

Engine lesson: Destroying a technology building may alter future production while the surviving army remains active.

Operation: Clones Down Under

all05umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civilian House (142)
- BadGuy1 House (76)
- Player House (35)
- BadGuy2 House (22)
- Temp House (4)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all05umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all05umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

6. Operation: Trick or Treaty

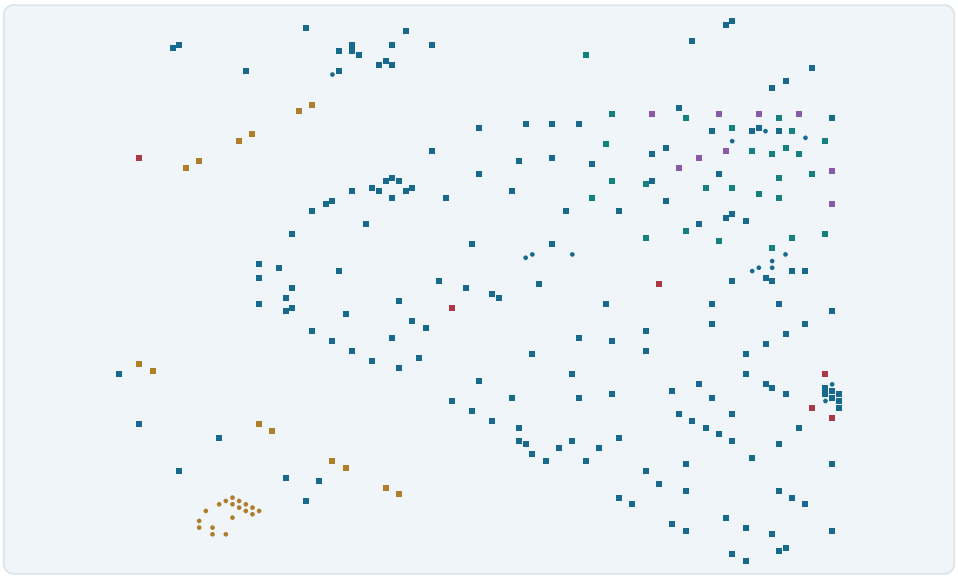
Protect the summit, then destroy its attacker.

1. Identify Parliament and the protected world-leader/summit scenario state.
2. Build a defensive economy and protect the approaches against Yuri's staged assaults.
3. Keep the protected objective intact through the defense sequence.
4. Move against Yuri's base after the scenario makes the offensive objective available.
5. Destroy the required Yuri base while continuing to avoid protected-object failure.

Engine lesson: A timed defense objective and a base-destruction objective can coexist, with explicit enabling links between their stages.

Operation: Trick or Treaty

all06umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (191)
- Americans (32)
- YuriCountry (27)
- Africans (9)
- French (6)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all06umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all06umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

7. Operation: Brain Dead

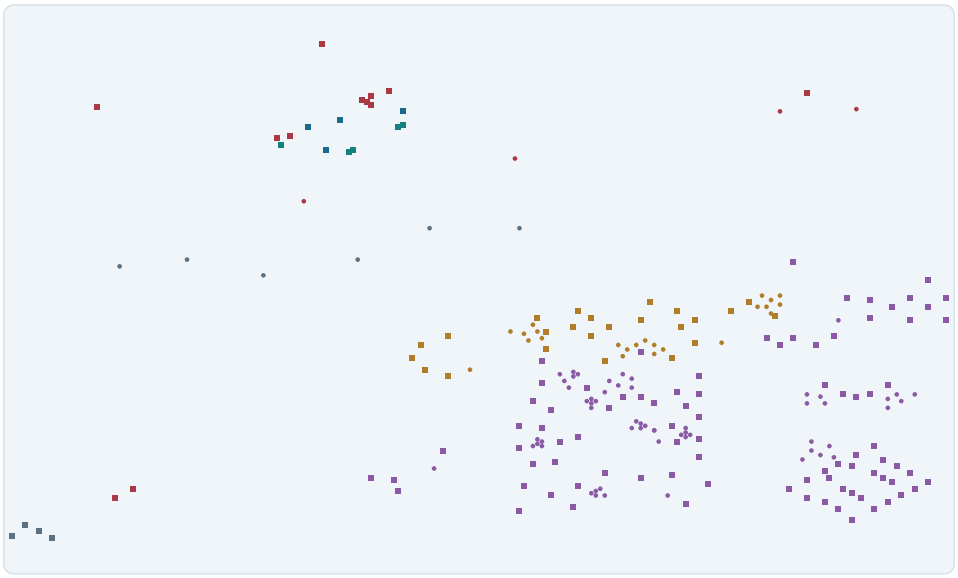
Recovered Soviet infrastructure enables a combined-faction finale.

1. Bring the abandoned Soviet base at Tierra del Fuego into a usable owned/operational state.
2. Build the Soviet Radar Dish required by the mission; this is a specific construction event.
3. Follow the resulting reveal/location sequence toward Antarctica.
4. Use the combined available technology to destroy the final Psychic Dominator.
5. Eliminate the remaining required Yuri forces and let the final outcome sequence complete.

Engine lesson: Captured/recovered technology changes the set of eligible production paths without merging the two games' entire rulebooks.

Operation: Brain Dead

all07smd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- YuriCountry (145)
- Yuri2 (47)
- Neutral (16)
- Yuri3 (10)
- Russians (5)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/all07smd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for all07smd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

Yuri's Revenge Soviet campaign

Stage	Mission	Detailed engine ledger
1	Operation: Time Shift	Open sov01umd.map
2	Operation: Deja Vu	Open sov02smd.map
3	Operation: Brain Wash	Open sov03umd.map
4	Operation: Romanov on the Run	Open sov04dmd.map
5	Operation: Escape Velocity	Open sov05umd.map
6	Operation: To The Moon	Open sov06lmd.map
7	Operation: Head Games	Open sov07tmd.map

1. Operation: Time Shift

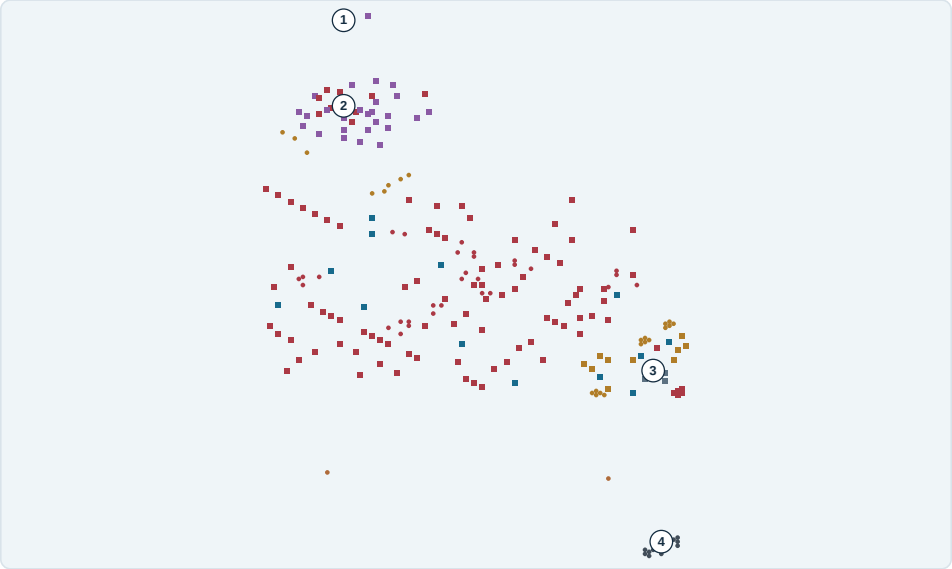
Several sequential objective gates control two different time periods.

1. Destroy the required Allied naval defenses and Grand Cannons around the approach.
2. Remove the Pill Boxes around the Time Machine so the scenario can transfer/control it.
3. Capture the required four power sources and defend the critical machine while the travel sequence proceeds.
4. Assume control of the available Soviet base after the scripted state changes, including the intermediate time-travel diversion.
5. Use that base/army to destroy the Psychic Dominator and satisfy the final mission gate.

Engine lesson: Time travel is a carefully staged map script with ownership, camera and object changes; it is not a universal ability every unit gains.

Operation: Time Shift

sov01umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (128)
- Americans (33)
- YuriCountry (28)
- Alliance (13)
- Soviet2House (13)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. YAPPET
2. YAPPPT
3. CATIME01
4. CATIME02

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov01umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov01umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

2. Operation: Deja Vu

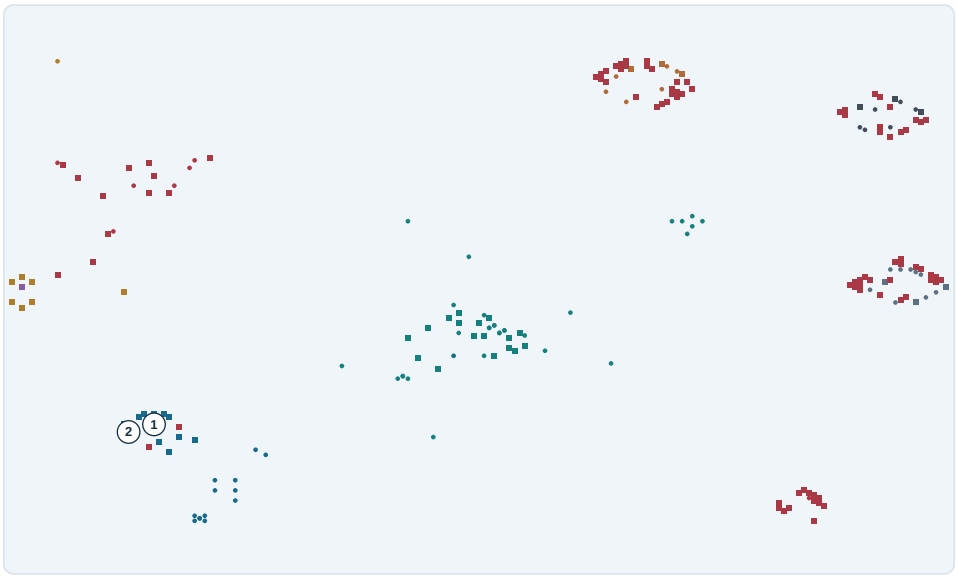
A revised attack targets both research and transfer technology.

1. Begin the return to the Black Forest with the available Soviet army and support.
2. Destroy the border outposts specified by the opening objective.
3. Develop the force and economy needed to reach the defended research site.
4. Destroy Einstein's Laboratory and the Chronosphere required by the mission.
5. Complete the combined objective flags and success sequence.

Engine lesson: Track the two objective structures independently; destroying one technology does not automatically count the other.

Operation: Deja Vu

sov02smd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Civilian House (95)
- BadGuy1 House (47)
- Einstein House (19)
- German3 House (12)
- BadGuy2 House (12)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. CALAB
2. GACSPH

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov02smd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov02smd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

3. Operation: Brain Wash

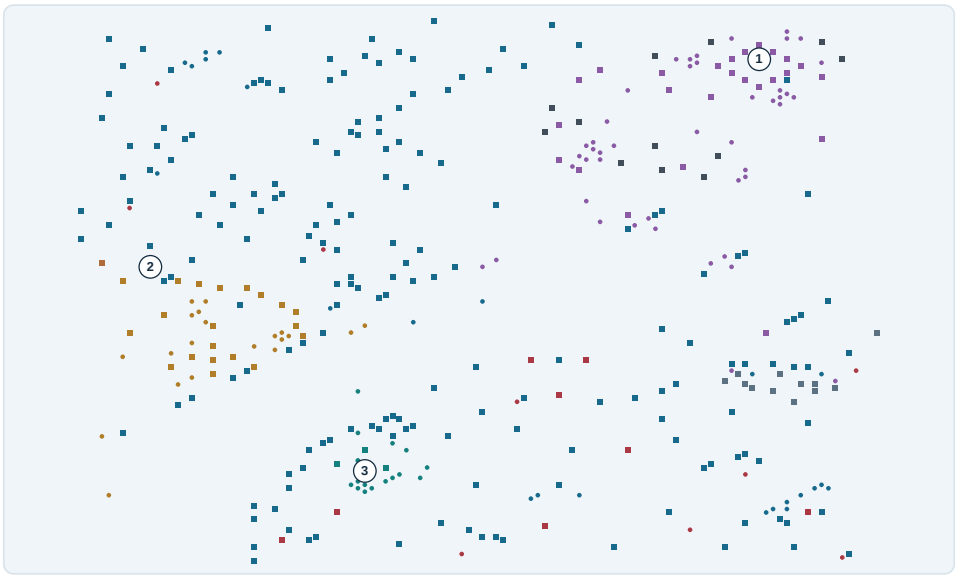
Free a captured army, then cooperate against the controlling enemy.

1. Advance the Soviet force toward the Psychic Beacon in London.
2. Destroy that beacon to trigger the release of the mind-controlled Allied strike force.
3. Use the changed ownership/alliance state and available forces against the near-complete Psychic Dominator.
4. Destroy the Dominator before its active threat/failure sequence succeeds.
5. Finish the remaining required Yuri forces and resolve the mission outcome.

Engine lesson: A mass ownership/alliance change is scripted; it must not be mistaken for every unit independently acquiring mind-control immunity.

Operation: Brain Wash

sov03umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Neutral (196)
- YuriCountry (71)
- BadGuy1 (41)
- Soviets (21)
- Civie1 (16)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Notable type references

1. YAPPET
2. NAPSYB
3. BORIS

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov03umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov03umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

4. Operation: Romanov on the Run

Escort completion requires both a safe destination and a living passenger.

1. Locate Romanov through the search/contact sequence.
2. Protect him as a critical unit while assembling a force to clear the extraction route.
3. Destroy Yuri's airport-guarding base as the objective requires.
4. Move the living Romanov to the now-valid airport extraction area.
5. Let the entry/evacuation sequence complete; clearing the airport without delivering him is insufficient.

Engine lesson: This joins a destruction condition, a protected-unit failure condition and a destination-entry condition.

Operation: Romanov on the Run

sov04dmd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- YuriCountry (101)
- Neutral (50)
- Alliance (19)
- Russians (9)
- Arabs (6)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov04dmd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov04dmd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

5. Operation: Escape Velocity

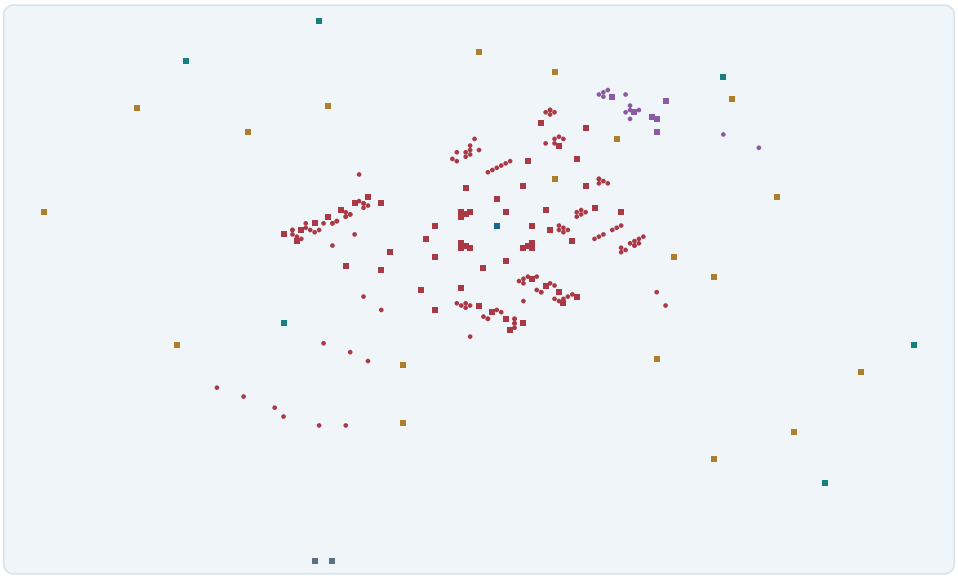
Choose a beachhead and remove naval production before the remaining cleanup.

1. Select a useful landing zone and establish a functioning Soviet base.
2. Prepare against the threat from Yuri's Boomer submarines and other scripted attacks.
3. Locate the specified Submarine Pen/naval production objective.
4. Destroy that pen and then remove the remaining required Yuri forces.
5. Complete the mission flags and transition sequence that prepares the lunar stage.

Engine lesson: Destroying a factory prevents/changes future production but does not itself erase already launched missiles or existing ships.

Operation: Escape Velocity

sov05umd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- YuriCountry (188)
- Arabs (19)
- Yuri2Country (18)
- Neutral (6)
- Germans (2)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov05umd.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for sov05umd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

6. Operation: To The Moon

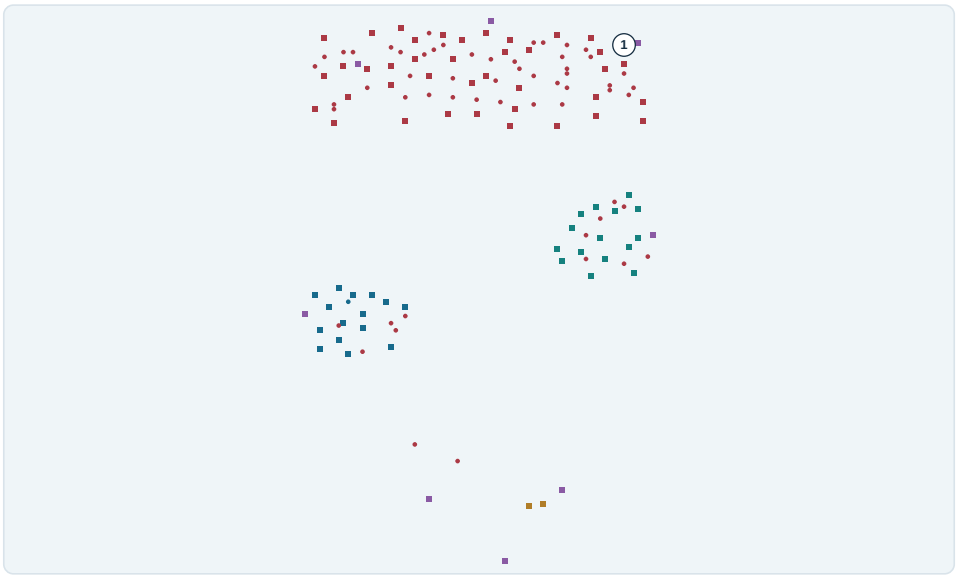
A different theater changes the available army and its movement context.

1. Find a legal site and establish the lunar Soviet base.
2. Use the mission's lunar economy, production and available Cosmonaut/vehicle options.
3. Locate Yuri's Lunar Command Center amid the moon bases.
4. Destroy that center and the remaining required base scope.
5. Complete the lunar objective flags and return/progression sequence.

Engine lesson: The moon mission uses a distinct map theater and local overrides; ordinary Earth skirmish assumptions are insufficient.

Operation: To The Moon

sov06lmd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- YuriCountry (97)
- Americans (16)
- Alliance (15)
- Neutral (8)
- British (2)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov06lmd.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. YACOMD

Initial placement schematic for sov06lmd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

7. Operation: Head Games

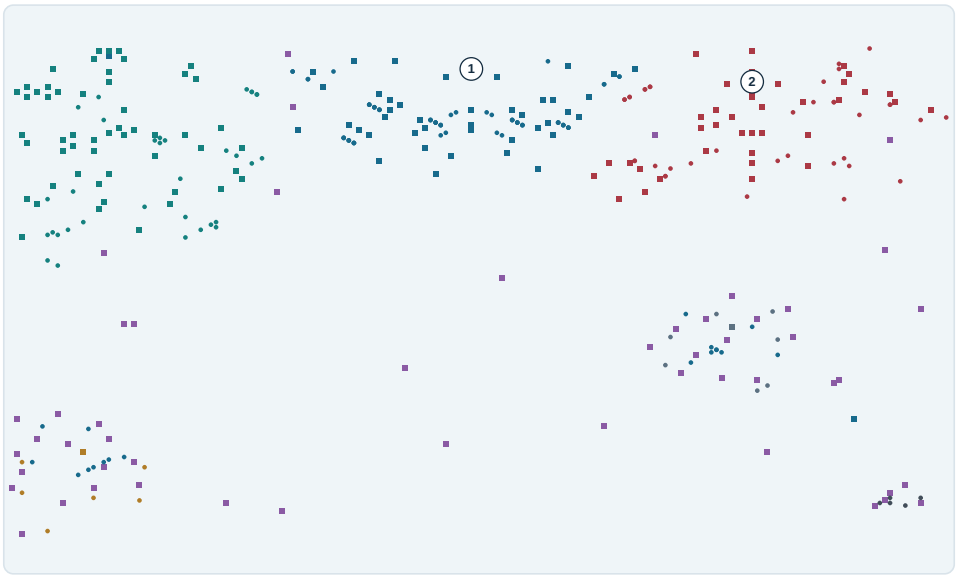
A fortified target is protected by several factions and scenario threats.

1. Establish the Soviet force in Transylvania and inspect the available captured/starting technologies.
2. Maintain economy and protection against Yuri's control effects and scripted enemy forces.
3. Break through the defended approaches using suitable anti-control, anti-air and anti-armor roles.
4. Destroy the designated Yuri Fortress objective.
5. Allow the fortress-destruction/finale actions to complete rather than treating every surviving enemy object as an unstated objective.

Engine lesson: The final fortress is a mission-tagged structure. Its story importance comes from the scenario network, not just its global health value.

Operation: Head Games

sov07tmd.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- BadGuy1 House (126)
- BadGuy3 House (86)
- BadGuy2 House (76)
- Civilian House (51)
- German2 House (8)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: mapsmd03.mix/sov07tmd.map; initial Structures / Units / Infantry / Aircraft records.

Notable type references

1. CATRAN03
2. NAPSYB

Initial placement schematic for sov07tmd.map. Explanatory diagram derived from the guide and local data; not a screenshot.

All conditions, teams and overrides | Exact outcome gates

Training and the additional recovered map

trn01t

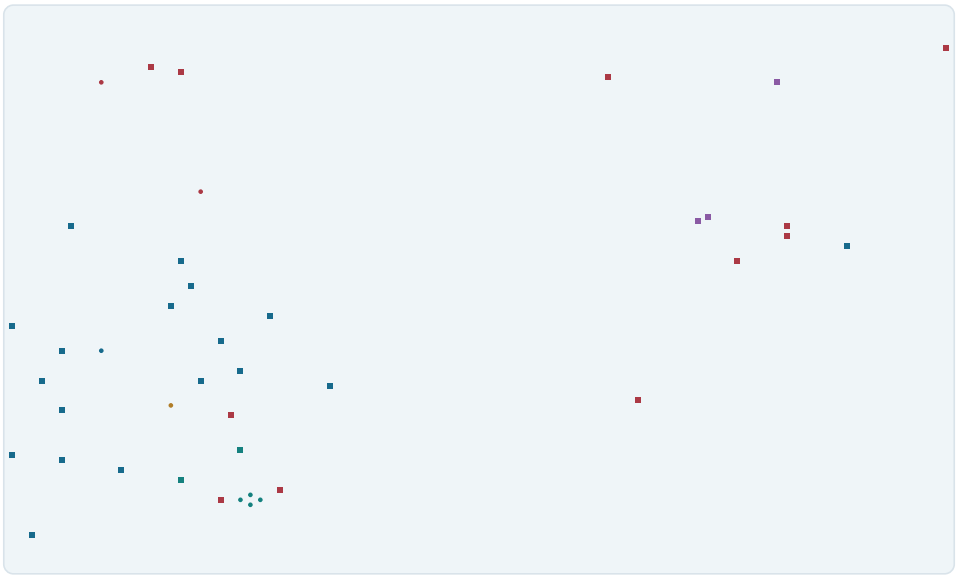
A sequence of instruction gates teaches direct control.

1. Let the instructor's opening sequence enable the first exercise.
2. Select and move the supplied units to the indicated area.
3. Perform the prompted attack and interaction exercises in their enabled order.
4. Use the local contact/selection/entry events to advance each lesson.
5. Complete the final training gate; later exercises need not be active before their predecessor is completed.

Engine lesson: A tutorial is an ordinary scenario with tightly controlled enabling and input/selection events.

Boot Camp - Day 1

trn01t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- Other1 House (19)
- Civie1 House (13)
- Other2 House (6)
- BadGuy1 House (3)
- Player House (1)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS01.MIX/trn01t.map; initial Structures / Units / Infantry / Aircraft records.

Initial placement schematic for trn01t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

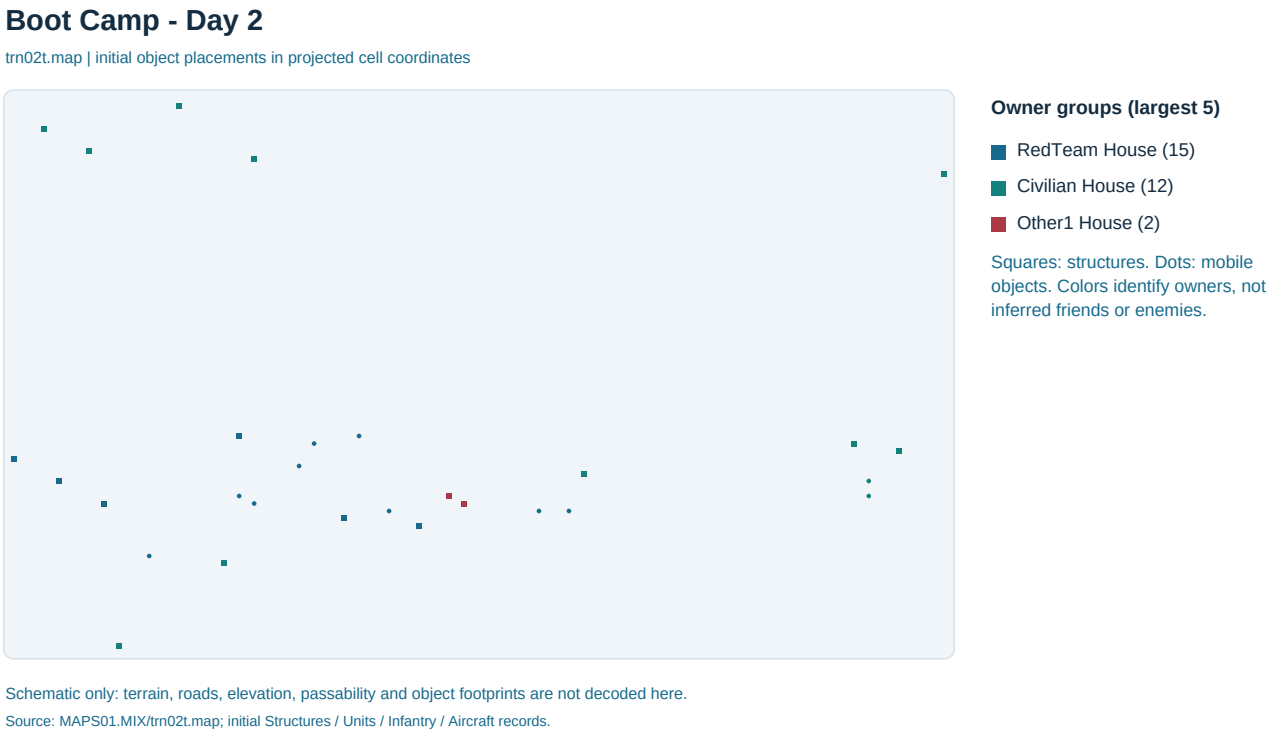
Full scenario ledger

trn02t

Base management is taught as a dependency sequence.

1. Follow the supplied construction/deployment instruction.
2. Establish the prompted power and economy facilities.
3. Produce the requested units through eligible factories.
4. Perform the indicated defense/attack exercises as their triggers are enabled.
5. Complete the final boot-camp sequence and proceed to campaign selection/progression.

Engine lesson: A build-type event can serve as a lesson-completion condition without destroying any enemy object.



Initial placement schematic for trn02t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

Full scenario ledger

sov09t

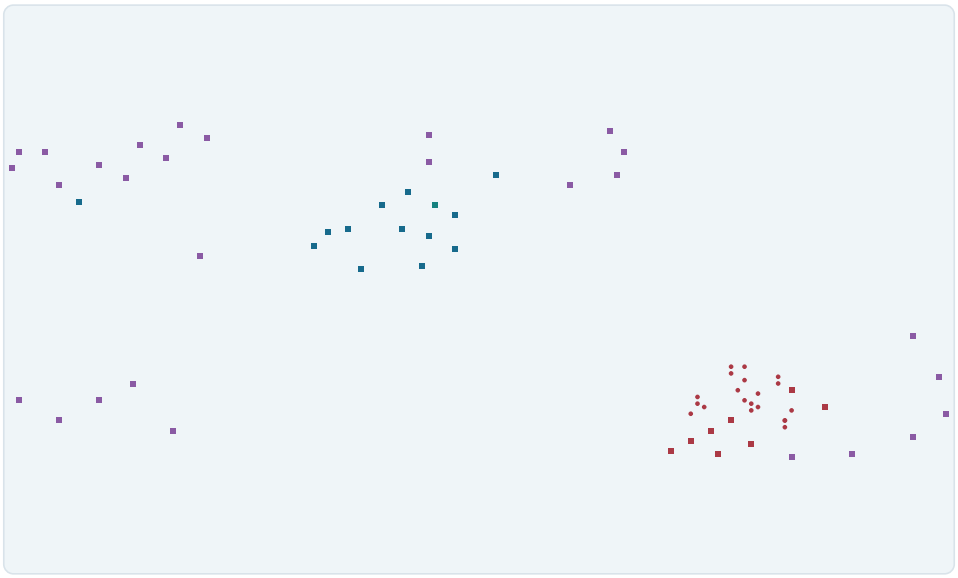
An additional older/alternate map is present but is not the campaign-selected ninth Soviet mission.

1. Treat this file as a separate recovered scenario, not a second mandatory stage after The Fox and the Hound.
2. Inspect its starting SovStar House ownership and its own task forces.
3. Follow its local trigger conditions and attached scopes when analyzing what it would do.
4. Do not transfer its short trigger list or starting army to sov09u.map.
5. Use the mission progression table's sov09u.map for the ordinary campaign route.

Engine lesson: A file existing in an archive does not prove that the campaign menu ever selects it.

sov09t.map

sov09t.map | initial object placements in projected cell coordinates



Owner groups (largest 5)

- SovStar House (28)
- Alamo (28)
- USA1 House (13)
- President House (1)

Squares: structures. Dots: mobile objects. Colors identify owners, not inferred friends or enemies.

Schematic only: terrain, roads, elevation, passability and object footprints are not decoded here.

Source: MAPS02.MIX/sov09t.map; initial Structures / Units / Infantry / Aircraft records.

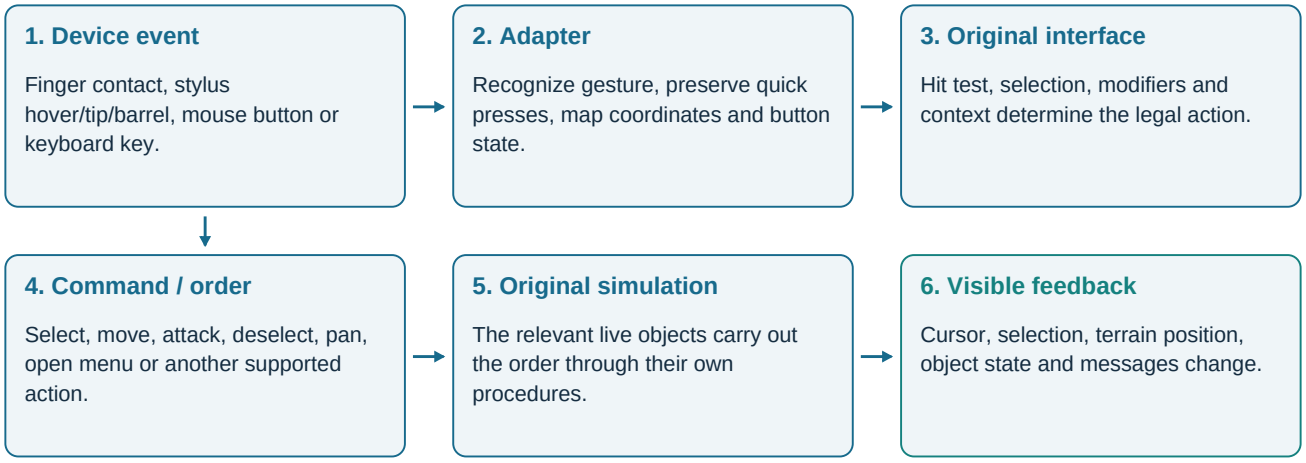
Initial placement schematic for sov09t.map. Explanatory diagram derived from the guide and local data; not a screenshot.

Full scenario ledger

How the Android version connects to the original game

Guide index | Player controls

From a physical input to the original command



Hover is movement without a click. Camera panning is different from ordering units to move.

Input conversion preserves the original command interpretation. Explanatory diagram derived from the guide and local data; not a screenshot.



Version 0.1.4 save dialog: the current mission precedes the complete local timestamp. Recorded RA2 save-name QA, injected finger input on an emulator.



Yuri uses the same mission-first naming bridge while retaining the original save interface. Recorded version 0.1.4 Yuri save-name QA on an emulator.

This chapter is specific to the delivered native development APK. It describes the local implementation and recorded QA, not a promise that every Samsung accessory or campaign has been tested.

1. Original logic on ARM64

1. Offline tools analyze the supplied original executables.
2. Recovered procedures are lifted into another compilable representation.
3. The Android compiler builds that representation as native ARM64 libraries.
4. At runtime, the host provides the operating-system services the original procedures expect.
5. The original initialized data/resources and packaged game archives provide required game content.

This is static recompilation with a compatibility layer. Preserving 32-bit guest addresses as values does not mean the APK executes the original x86 instructions. It also does not mean the recovered representation is the original developers' handwritten source or that every function has a trustworthy descriptive name.

2. Input conversion

1. Android reports the input device and event: finger, pen, mouse or keyboard.
2. The adapter recognizes the intended gesture/button combination.
3. Coordinates are mapped into the game's displayed picture.
4. Mouse movement/button state or a keyboard event reaches the original interface.
5. The original game decides selection, commands, deselection and map panning.

For fingers, a single contact is held provisionally until movement/release distinguishes a selection from a multi-finger chord. This avoids accidentally issuing a left click while preparing a two-finger right click. Two-finger movement holds right for panning; lifting either releases it. Three-finger quick taps send Escape. A drag selection followed by extra fingers is not reinterpreted as a fresh quick tap.

Pen hover forwards cursor movement without pressing a button. Pen tip and barrel events use the mouse-button path. A physical OTG/Bluetooth mouse follows the same full-button-state path. The toolbar can hold right for a pan or provide keyboard-like commands. See INPUTS for the exact player-facing controls and limitations, including wheel behavior.

3. Preserving quick presses

1. Android can deliver press and release within one native event-pump interval.

2. If the game polls only after both, a naive adapter can make the button appear never pressed.
3. The local adapter retains a bounded down state so the guest can observe a quick click.
4. Button states are tracked independently.
5. Cancellation or pause releases held buttons and modifiers to avoid stuck inputs.

This changes event delivery, not the meaning of a tank attack or the underlying right-click command.

4. Saving without a keyboard

1. Open Save Game in the original mission menu.
2. Let the original dialog initialize its current mission title.
3. Read that title through the original custom Unicode edit-control messages.
4. Append the full local date and time, keeping the mission title first.
5. If needed, shorten only the title to fit the original 79-UTF-16-unit capacity.
6. Write the resulting name back to the original edit control.
7. The original save procedure writes the save file.
8. An Android confirmation bridge sends the original acknowledgement back on the game thread.

The application does not require a soft keyboard for this flow. Existing saved labels are not renamed. Recorded version 0.1.4 tests checked formatter edge cases, RA2 and Yuri saves, restart/load, continued simulation and preservation of five pre-existing manual saves.

5. Movies, sound and presentation

1. The original game requests its movie or sound.
2. The host provides the relevant decoding/audio service.
3. FFmpeg decodes the original Bink movie content in this implementation.
4. SDL supplies the Android surface, event and audio connection.
5. End-of-file, seeking and decoded-frame timing have to agree with the game's polling expectations.

The local movie-end fix prevents a completed clip from trapping the game in a repeated final-frame poll. Full-screen movie Escape handling has recorded control QA. In-mission radar-movie skipping and audible output on each physical Samsung device are separate questions; the book does not claim they were all verified.

6. Returning home

1. The player uses the original Exit Game path.
2. The game requests a normal process exit.
3. Before that exit, the host brings the title chooser forward.
4. The isolated game process terminates.
5. A later title launch starts with a fresh game process while the chooser remains available.

This architecture protects the chooser from the original program's process-lifetime assumptions. It is different from returning only to the game's own main menu or leaving the Android app entirely.

7. What the recorded checks establish

Evidence	What it supports	What it does not establish
Effective archives and map extraction	Rules, declared objects, trigger data and content provenance	Every map branch is reachable or correct in a real run
Function and class catalogues	Recovered binary structure and identified responsibilities	Complete recovery of original source-level intent
Existing campaign startup captures	Particular missions reached visible gameplay	Completion of all campaigns
Existing input injection checks	Tested adapter paths in their recorded environment	Physical S Pen/OTG behavior on every Samsung setup
Version 0.1.4 save/reload QA	Specific original save flows work across restart	Every possible saved state is compatible
This guide's automated coverage check	Declared items/mission records are represented and references resolve	A substitute for playing every mission

No new full campaign playthrough was performed to write this guide. The study exercises explain how to compare the documented logic with your own play without confusing predictions with observations.

Reading while playing: small engine experiments

Guide index | Controls



Use a small, recognizable group of objects for controlled observation. Upstream project screenshot retained locally; illustrative desktop evidence, not an Android-device test.

These are proposed observation exercises, not tests already completed by the author. Use a fresh disposable save before an experiment. Keep your important saves separate and use the correct game title when loading. The current APK fills new save names with mission and time, so you can distinguish checkpoints without typing.

1. Trace one GI

1. Select a GI in a safe location.
2. Move it and observe its mobile state.
3. Deploy it, then compare the permitted movement and attack state.
4. Undeploy and move again.
5. Where a suitable civilian building exists, enter and then unload.
6. Read the GI card and distinguish its standing, deployed and occupant weapons.

Record which action you requested and which state changed. Do not infer that the deployed weapon and garrison weapon are identical just because both produce heavier fire.

2. Trace an IFV and a passenger

1. Obtain an empty IFV and an Engineer in a suitable battle.
2. Observe the IFV's empty role.
3. Load the Engineer and observe the changed role/cursor/turret.
4. Use it on an eligible damaged friendly vehicle.
5. Unload and observe the return to the default role.
6. Compare with a different eligible infantry passenger and its IFV mode on the card.

This demonstrates a state-dependent vehicle configuration. It does not establish every possible passenger combination from one example.

3. Follow a resource cycle

1. Choose a safe miner and identify its refinery/delivery destination.
2. Follow the outward journey to the resource.
3. Watch cargo collection and the return procedure.
4. Observe unloading and the owner's credits.
5. Repeat with the other faction's miner in another battle.
6. For Yuri, compare the mobile Slave Miner, deployed form and worker trips.

Keep transport time, gathering time, docking time and delivery value as separate observations. Comparing only vehicle speed cannot establish complete income efficiency.

4. Observe power and services

1. Find a battle with a powered defense or Robot Control Center.
2. Save before altering the power balance.
3. Note current supply/demand and which services are working.
4. Reduce power through an ordinary available game action.
5. Observe which services stop, slow or remain functional.
6. Restore power and compare recovery with the structure/unit cards.

Do not destroy an irreplaceable campaign objective just to run this exercise. A quiet skirmish is a better setting when the required facilities are available.

5. Trace Lone Guardian's objective handoff

1. Read the mission explanation and ledger.
2. Identify the opening Dreadnought/Tanya/landmark roles.
3. Observe the first objective without assuming all later hints are active.
4. Reach Fort Bradley and note changes in ownership, revealed terrain and available production.
5. Follow the bridge/production hints to the later supply-base objective.
6. Compare each observed transition with its trigger's events, attachments and actions.

A camera movement is not the same as a new unit spawning. A revealed building may have existed from the start. Watch these separately.

6. Compare two kinds of capture objective

1. Read Big Apple's Engineer/Battle Lab progression.

2. Read The Fox and the Hound's President-control progression.
3. Identify the different specialist and target-eligibility checks.
4. Observe the required ownership/control result in your play.
5. Compare with a mission that explicitly asks for destruction.

The practical lesson is that capture, infiltration, mind control and destruction are four distinct interactions even when each can remove an enemy obstacle.

7. Observe a staged attack

1. Save before a known wave interval or mission boundary.
2. Observe where the arriving force appears and which units it contains.
3. Find its reinforcement/create-team action in the ledger.
4. Follow the linked roster and script.
5. Compare staging, movement, unloading and final attack with the plan.
6. If something differs, check difficulty, earlier flag changes and unit losses before concluding the data is wrong.

Do not expect a static file listing to prove the exact random choice or timing of every run. Record the conditions under which you observed it.

8. Check the input-to-command boundary

1. In a safe area, move a mouse or hover the S Pen without pressing.
2. Observe cursor movement without a unit order.
3. Select with a left click/tip/finger action.
4. Deselect with a short right click or quick two-finger tap.
5. Pan with right-drag or two-finger drag.
6. Open the menu with Escape or a quick three-finger tap.

This exercise separates the Android adapter from the original command interpretation. Record the device and peripheral used; emulator injection does not qualify the physical accessory.

9. Read a save/reload as a state-restoration experiment

1. Stop at a recognizable, safe moment and make a new save.
2. Record a few visible facts: units, health, location, credits and current objective.
3. Change the battle briefly.
4. Load the saved game from the corresponding original menu.
5. Check that the earlier state returns and the simulation continues advancing.
6. Distinguish a changed save description from a changed save-file format; this APK's naming bridge leaves original serialization in charge.

Observation sheet

Record	What to write
Game and mission	RA2 or Yuri; mission title; difficulty
Starting checkpoint	Automatically generated save label
Device/input	Phone/tablet, finger/pen/mouse and relevant accessory
Requested action	What you deliberately did
Prediction	Which card or trigger suggested the expected result
Observed result	Visible state changes, messages, ownership, timing or failure
Source locator	Object ID or trigger ID from the guide
Remaining uncertainty	What this one observation did not establish

Keep predictions and observations in separate columns. That is the most useful habit for learning an engine from binaries and play.

Images, screenshots and their provenance

Guide index

The screenshots below are existing project captures copied byte-for-byte. They are not newly recorded complete playthroughs. Diagram figures are separate authored explanations. Mission layout figures use initial placed-object coordinates; they do not decode terrain or prove reachable paths.

ra2-allied-start.png



RA2 Allied opening: Lone Guardian. Dreadnoughts, the river approach and the original sidebar are visible. Historical local Android-emulator campaign-start capture; not a complete mission playthrough.

Source in original workspace: Native-APK/qa/ra2-allied-mission.png. SHA-256: 9eb4c15dfe5202c95dc44d1b1dc2dbeb69b9262ba46bad7a004e23ba38f712ff.

ra2-soviet-start.png



RA2 Soviet opening: Red Dawn. The initial Washington scene is visible with the Android command toolbar. Historical local Android-emulator campaign-start capture; not physical Samsung testing.

Source in original workspace: Native-APK/qa/ra2-soviet-mission.png. SHA-256: 399cf758b9c2b714dab7cdd85c1a86190334a3b5182bda15a841ea76560b86bd.

yuri-allied-start.png



Yuri Allied opening: Time Lapse. The starting base contains multiple building roles and a visible Dominator countdown. Historical local Android-emulator campaign-start capture; not a completed campaign.

Source in original workspace: Native-APK/qa/yuri-allied-mission.png. SHA-256: 8f7a70bb6ab11438cdecfc935f5f6306ef0971f18c40daa203a666637ad094da.

yuri-soviet-start.png



Yuri Soviet opening: Time Shift. Naval forces and the defended shore show why target domains and arrival scripts matter. Historical local Android-emulator campaign-start capture.

Source in original workspace: Native-APK/qa/yuri-soviet-mission.png. SHA-256: ffd4a96e17bf0fadd3d6967e07a6d92af5fc66bea7b7fd190c8590c40eda7ec4.

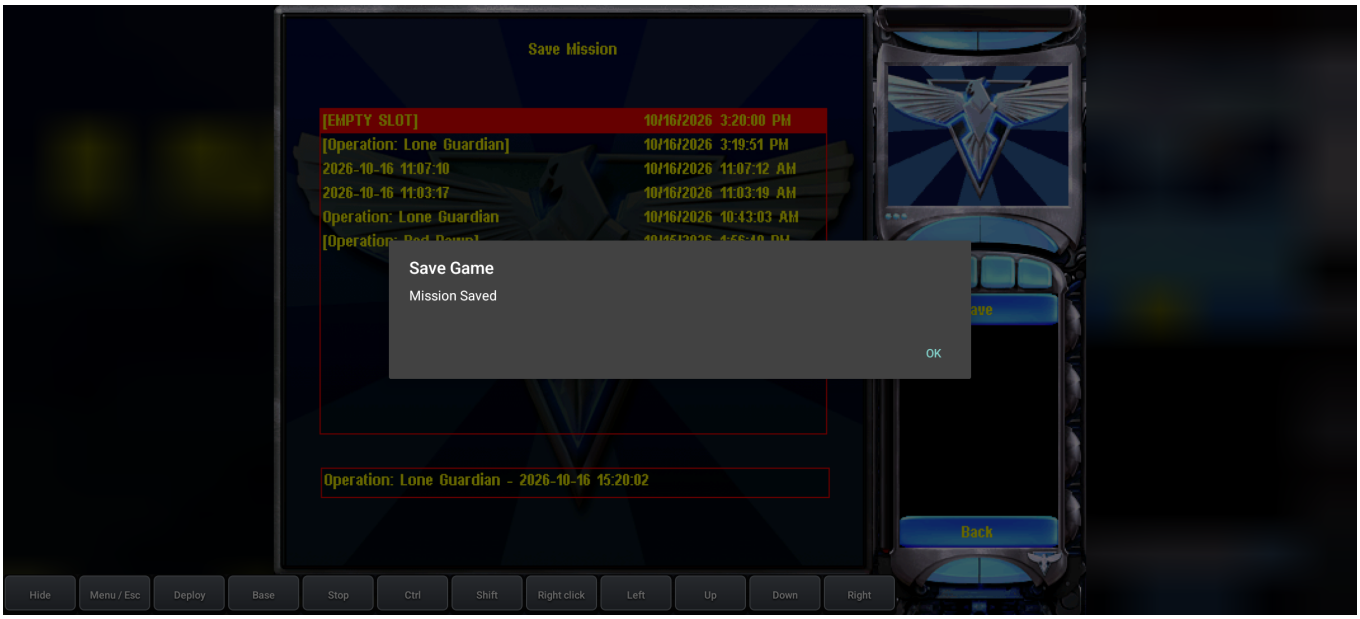
save-name-ra2.png



Version 0.1.4 save dialog: the current mission precedes the complete local timestamp. Recorded RA2 save-name QA, injected finger input on an emulator.

Source in original workspace: Native-APK/qa/save-0.1.4/classic-finger/save-name.png. SHA-256: cad32c48726286c007ab00bd99731d955337a985682816cd5990a6a953ba3e78.

save-confirmation-ra2.png



The original save flow reaches its acknowledgement through the Android bridge. Recorded version 0.1.4 save QA; read with its report and limits.

Source in original workspace: Native-APK/qa/save-0.1.4/classic-finger/save-confirmation.png. SHA-256: c85b8d8011ee855337368ab0681ab3b81dc2d06620c506a5b4e9c10643597d88.

save-name-yuri.png



Yuri uses the same mission-first naming bridge while retaining the original save interface. Recorded version 0.1.4 Yuri save-name QA on an emulator.

Source in original workspace: Native-APK/qa/save-0.1.4/yuri-finger/save-name.png. SHA-256: 2c6aac384ff4d528acef3a06727d966e8bc1d400f8d63677c16a4d5467dffff4.

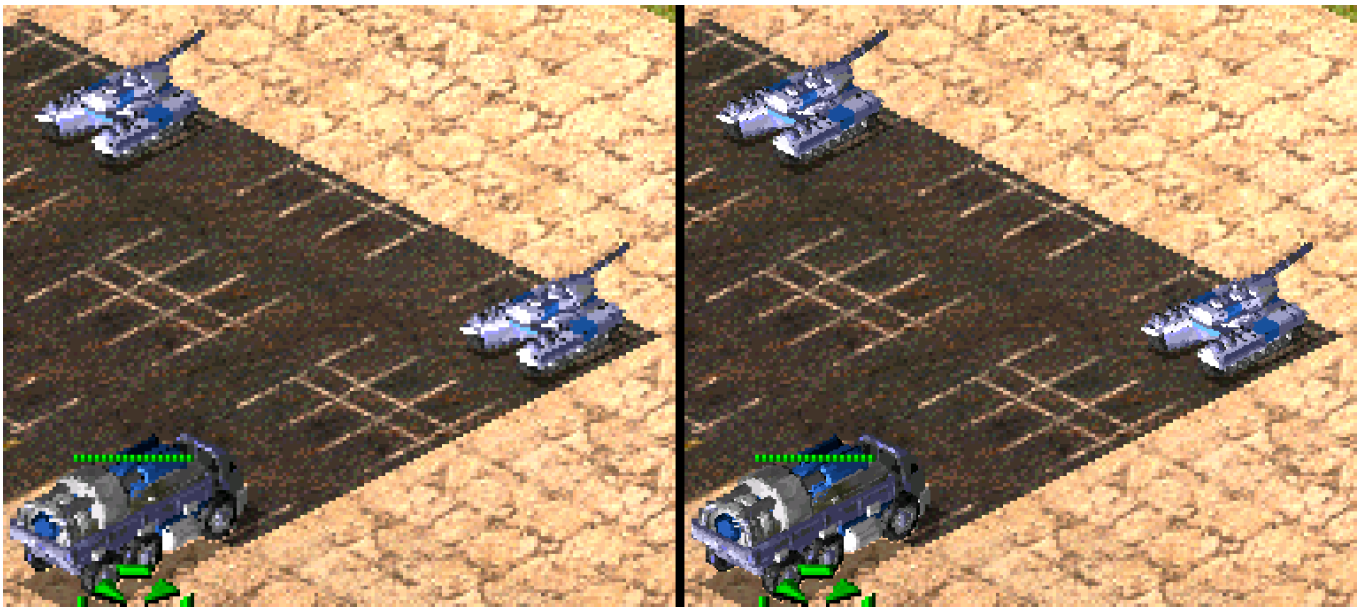
unit-roster-desktop.png



A supplied desktop-runtime capture shows mobile objects sharing a battlefield: the selected MCV, tanks and infantry. Upstream project screenshot retained locally; illustrative desktop evidence, not an Android-device test.

Source in original workspace: native-ra2/docs/screenshots/skirmish-units.png. SHA-256: a78c7dcdc53fd8aeffa824148c17fddfbabc0caa2d7e2b569e41dcd635af5c89.

voxel-comparison.png



The local renderer notes compare normal and enhanced voxel presentation. Read this as a rendering example. Supplied upstream desktop comparison; does not establish that the Android APK enables this enhancement.

Source in original workspace: native-ra2/docs/screenshots/hd-voxels.png. SHA-256: 7329b4975ae22bde3118ef2b0e6727b5b307393194b851c283836975c2a902ac.

Diagram sources

Eight workflow diagrams explain engine layers, unit decisions, combat, economy, buildings, triggers, AI teams and Android input. Forty-one placement schematics derive from the recovered scenario maps. Owner colors distinguish source owners; they are not an assertion of allies or enemies. Initial placements do not include units later created by triggers.

Sources, verification and limits

Guide index

Prepared from the local game install and port workspace on 26 October 2026. This is a plain-language analysis, not the original source code and not a claim of a completed playthrough of every mission.

Evidence labels

- **Data-confirmed:** present in extracted rules, campaign tables, maps or strings.
- **Binary-supported:** backed by local class/function recovery or inspected lifted procedures.
- **Manual-described:** player-visible behavior described by the supplied English manuals.
- **Conceptual reconstruction:** an explanatory decision sequence consistent with these sources, without a claim that every original instruction/order has been recovered.
- **Unverified in play:** a prediction or branch not exercised in a recorded playthrough.

Extraction and precedence

1. Read the original MIX archive index, decrypting its header where required.
2. Resolve member names with the game-compatible filename hash.
3. Check every member span stays within the declared archive body.
4. Extract the known rules, AI, campaign tables, localization tables and campaign maps.
5. Validate the complete CSF string-table structure and decode its text.
6. Apply the patch archive's rulesmd.ini and all02umd.map in preference to the base members.
7. Keep RA2 and Yuri rules separate.
8. Merge a mission's explicit overrides only for its scenario; missing values are not invented.
9. Translate every recovered event/action record, retaining action order, identifiers and source line locators.

The tools and raw extracted resources remain in the local build-cache/engine-guide-research folder. The reading folder contains derived explanations and factual reference tables, not a new dump of the original rule/map files.

Coverage

Item	Count
Distinct registered mobile types across both games	157
Distinct registered building types across both games	408
Story missions	38
Training missions	2
Additional legacy/alternate map	1
Documented scenario maps	41
Translated trigger definitions	4274
Translated event records	4787
Translated ordered action records	13724
Weapon/effect cards (game-specific)	341
RA2 global AI team templates	103
Yuri global AI team templates	163

Local binary evidence

- native-ra2/game/game.exe: 5,077,312 bytes; SHA-256 73288c03b58d370be268ca6d156b4e33bfdb2066dc980359467d8852ff3b00df.
- native-ra2/game/gamemd.exe: 5,286,208 bytes; SHA-256 3e81a61775d2745d1dabe397325ef663cd994ffc194da4e998e3bf5d2d308600.

The local recovered class catalogue contains the expected live-object/type, house, cell, trigger, team, factory, projectile and movement-controller responsibilities. The guide does not equate class names with a complete recovery of their algorithms. The Yuri function catalogue and lifted action runner were checked together. The action-runner trace at original address 0x007265C0 checks enabled/destroyed state, walks the action list and invokes the executor at 0x006DD8B0. Those addresses are version-specific evidence locators, not mobile gameplay instructions.

Local source locators: native-ra2/work/rtti.json; native-ra2/work/game/rtti.json; native-ra2/work/functions.json; native-ra2/src/recomp/gen/recomp_0049.c; native-ra2/docs/voxels.md; native-android/PORTING.md; native-android/src/android_entry.c; native-android/app/src/main/java; Native-APK/qa/save-0.1.4/report.md. Relative paths here describe the original workspace and are not required to read the published guide.

Supplied manuals

The accompanying English RA2 manual explains units, structures, garrisons, Engineers, economy, powers and controls. The English Yuri manual explains expansion changes and the added units/structures. They were text-extracted locally for this analysis; the reading guide paraphrases their gameplay explanations and cross-checks explicit rule fields. Manuals can describe intended behavior that differs from a particular patch or scenario.

Local files: Command.&.Conquer.Red.Alert.II/game/Manuals/Red Alert 2 - English 2 - Manual.pdf; Command.&.Conquer.Red.Alert.II/game/Manuals/Yuri_s Revenge - English - Manual.pdf.

Primary technical cross-checks

- YRpp TriggerClass: event state and trigger-action responsibilities.
- YRpp TActionClass: action executor and identified operation entry points.

- YRpp ScriptClass: ordered script state.
- YRpp GeneralDefinitions: event identifiers and object-mission identifiers.
- FinalAlert extension documentation: editor labels for script instruction numbers. Only vanilla instructions used by these maps are described; extension-only features are not assigned to this APK.
- The supplied FinalAlert2/FAData.ini event/action descriptions supply the main translation vocabulary. Its modification date is newer than the retail game, so descriptions are cross-checking aids, not proof that later extension behavior is present.
- Static recompilation project: background for the local runtime and renderer research. Android behavior here is established from the local port, not inferred from desktop feature claims.

What remains open

The guide does not claim exact pathfinder search/tie-breaking, every target-score formula, complete save serialization internals, exhaustive numeric hardcoded defaults, or proof that every declared trigger is reachable. Special editor parameters not independently established remain named configured values. No new complete campaign playthrough, physical Samsung S Pen/OTG qualification, performance measurement or long-session stability trial was performed while writing this guide.

The legacy sov09t.map is documented separately from the campaign-selected sov09u.map. Both tutorial maps are included. Multiplayer/co-op/skirmish maps are not each treated as a new story mission, and multiplayer is disabled in the delivered APK. Global rules/AI still explain their shared mechanisms.

Extracted member fingerprints

These identify the exact local source bytes used. Duplicated maps in multiple archives remain listed so provenance is not lost.

Archive/member	Bytes	SHA-256
ra2.mix/local.mix/rules.ini	541915	fd1e95cea0306ea78049dc81c8cd816e18c28c496872a1ff02edd50bd082062f
ra2.mix/local.mix/art.ini	245825	b477f861063a9509e87b7836b78190204c73bea5ea5595a842c456f5920ea223
ra2.mix/local.mix/ai.ini	84972	1feac6ddea6886b177ddf7e5f8580b7a99a63f12684f2cbb42831671bb7a8a79
ra2.mix/local.mix/battle.ini	4054	a5752a8d49e1c3823fe01a55e0057f91f2996d8b4a78804f197251a79b552a1d
ra2.mix/local.mix/mission.ini	9756	49eb674e46058d67a08e0cf4ba31c99b1948d3d2072b10beced873f62226c54e
ra2.mix/local.mix/mapsel.ini	7875	f4bcbb678d182c35a4331d69c92150a719d73fc0c95fae36cc52e65d74923982
ra2.mix/local.mix/tutorial.ini	14762	59b44499e1720c275e78cb5397e19fb94dbac46e22ea54c220e01e9e1a1d0492
ra2md.mix/localmd.mix/rulesmd.ini	742958	06761dd7f714e7d9400216ec3c06109ec5c1461f6a0727be7401eb9d8b0f6d05
ra2md.mix/localmd.mix/artmd.ini	336535	e1f0378394313c04ebbd5073f47785ee3e46f1b3c62d65724e8f3c310ee7ba31
ra2md.mix/localmd.mix/aimd.ini	138538	5df41eaec00a78d0760ef5eecdf27d65ae1cd537309c7eac973318266986f89d
ra2md.mix/localmd.mix/battlemd.ini	2733	7e40691a3480b1e5ffe6d1cdfccb83f67da4447157607ec3ecb5d2278acf0791
ra2md.mix/localmd.mix/missionmd.ini	5034	ce170ec97fc28589ee971792a31d135e0f0510f878acde3fd0d79a7b7896a775

Archive/member	Bytes	SHA-256
ra2md.mix/localmd.mix/mapselmd.ini	5573	074ccccd9e156d586d0ed099ceab78b3d72384c808725d8b5bee2c9799691d52
expandmd01.mix/rulesmd.ini	743215	3d341ef8a13a4b5ab24af2eef48ac94931ac2bb87d950fe3330a07e2d25672ef
expandmd01.mix/all02umd.map	465788	e761d28ce43401308ca4532d65479b1cf4a309d74fd4f996fafa92a0a06f8b65
language.mix/ra2.csf	485776	e46715fcdf038c4c3231a457873fc711174110d721a0a5390a508dbadcf4ff72
langmd.mix/ra2md.csf	573269	f3382231d7eb3bd713bf3908ecdba175a1b2e20d9d80fa2702e9ab84c3a79deb
MAPS01.MIX/all01t.map	146240	ad64c9293a8bccb85c38f5871a974ed9c09b8b5da11bb346e990423bf625c79c
MAPS01.MIX/all02s.map	191242	5ebd7a3aaa60215a6da1f63e87a53c4ebf8cb28378b09f7128953ab05e6425fc
MAPS01.MIX/all03u.map	202576	186e317d4be77eaeec8e0b09e5c813f6ff95aa01e4e206f4b4c82178bf679a61
MAPS01.MIX/all04u.map	222831	09e52db16946a330c032e988679ef61ff7745670ddad5e9bfcf96f916d18d093
MAPS01.MIX/all05s.map	214744	d09cebddc1f4009098dfa48b18bf4022e6e4d00801f1ca8efd99917f185bb225
MAPS01.MIX/all06u.map	260458	9e02b8fa4c570000c2bb34c9b0e702b143e2e93ae529faa36556e1f43af0f922
MAPS01.MIX/all07t.map	471258	3925c1211d86938a7137ff0df11cba85c59d3ae75742221d5e2d4d502d33c103
MAPS01.MIX/all08u.map	199442	71a119187bd8f78a1c7677636e20a18a258f24dc94add2ac2b108a3dae9df8ea
MAPS01.MIX/all09t.map	190682	89a243ea0d9ccc802bd6e1d7da4d271d0077f86c3883802e27be7840364ac279
MAPS01.MIX/all10s.map	289946	ac0ea14ec650256a014626db7c78135bb74168ba86d4eb863d987f04b07d1593
MAPS01.MIX/all11t.map	369324	08456515dd5e258f5a9ff73b8961491883eccaa3635a8f008dcb1f9369d43887
MAPS01.MIX/all12s.map	185688	e6d31134742f8566a425a252113fa6b8e3f3eb178b00e34124deec4f7e11250b
MAPS01.MIX/sov01t.map	184260	7f4492961d30b5799367aa840ba007932057b19a22a77f95475a18ca1407b99f
MAPS01.MIX/trn01t.map	98389	4dcec9e12f2541e4c15bf1e7da78c6ff74932c794837c35029a02de5bf43cd71
MAPS01.MIX/trn02t.map	57795	35f7b531a8d0312ca7e44fa1da33f88dde1af48ac3781841bc74d3122c498445
MAPS02.MIX/all01t.map	146240	ad64c9293a8bccb85c38f5871a974ed9c09b8b5da11bb346e990423bf625c79c
MAPS02.MIX/sov01t.map	184260	7f4492961d30b5799367aa840ba007932057b19a22a77f95475a18ca1407b99f
MAPS02.MIX/sov02t.map	186385	ea3aff6876f7e81e76284a2c9b41ee693c71f0f27e1ae45542e8f5ea95c96ae7
MAPS02.MIX/sov03u.map	173904	a0ec1cd648f2b1f5cd122713d405b514634941977289080e22990021f8719c05

Archive/member	Bytes	SHA-256
MAPS02.MIX/sov04s.map	201817	a0a95b26639cc1d4d916dea3a264a18e409bef3d2b84c245e67cd250defc2100
MAPS02.MIX/sov05u.map	85694	31c6883ce678c0bcede293c571a610d355319a7cd729d45560183ced152a5629
MAPS02.MIX/sov06t.map	262145	bea4548cca589aa1caf6fbc4e69700f8b408d650b700b71c65242f0df9eeeb83
MAPS02.MIX/sov07s.map	178521	0ad2fde5c76f17085065f7b43cb76045935774092b2498badc44fe91e7793171
MAPS02.MIX/sov08u.map	197257	91817fa1585e631cd929b6356ccc29c99f1dfb8144c52cd225e3eeb924370b7e
MAPS02.MIX/sov09t.map	110638	e444665c9afec8ffa2528d4e10641739f8a7c16921522b91dba3520cacd71041
MAPS02.MIX/sov09u.map	105691	ef2dd5c5280d20f5759c02cebf860111280dc0f091c322f21b42eeae6b53c8b6
MAPS02.MIX/sov10t.map	276498	5c3d8b26e05a0a73530630db8353637fa9ed7072144d5360e1ab994869428449
MAPS02.MIX/sov11s.map	187412	2f51c08a063f3c4d648bd486672eacc50d3f32903aa14e9566bfd2030464fc7f
MAPS02.MIX/sov12s.map	231726	f69c18a54ce5915c9cdc08c2f10113853bc8656cf3e5f1392a64818a700d2720
mapsmd03.mix/all01umd.map	301077	dee38769f2247a85908705486c175ef14a4cf90437899defe7c8b4c0d1c51fb0
mapsmd03.mix/all02umd.map	465825	168ed42024991b6c2af4de0e5a667e9c2bf15767741efe99dac975721460b303
mapsmd03.mix/all03umd.map	418628	b84a2e689cc00f0be064ecbfcd385635ac037f28bf8f72c02b426a0e52ff1547
mapsmd03.mix/all04dmd.map	282057	f1621a21e7221c1e475a52dccd4a04bb3125f8a095377c917e2122019f2157b2
mapsmd03.mix/all05umd.map	274279	7ee8148ad4b0092b7970588d9b064ddd6b753fc7bfb352bcc043affeb72c6aed
mapsmd03.mix/all06umd.map	192301	cfd2b50c5448fd84ab3647ff3a95f96b45e8dccd6a72b2601c0bb603cddeeb63
mapsmd03.mix/all07smd.map	434373	0ec687c320382e4c02cf66eec0153b713e56c4449fab7d29b5a19b61bf10e4f3
mapsmd03.mix/sov01umd.map	329188	498b989a4b6f579dee0904a77f1fabf982c14786db21a46c41900fc8344ec72b
mapsmd03.mix/sov02smd.map	327808	139a7f17a17cefc20d12c21e9221d94ff6e1e8894ac09ab716fc9f6975d5a665
mapsmd03.mix/sov03umd.map	189836	df69d0f63f982823bb5818d3d49776028a719a2c20ced2526fab8f5cecb49fe6
mapsmd03.mix/sov04dmd.map	228003	702bd788efbd08ae5b7591a1dfc240684e73faedace2ffaac68baefdfd0fb12a
mapsmd03.mix/sov05umd.map	467636	22518e8b4a86331a9172048e10de13b45c2826ea4fdf4d2203b58994b897761b
mapsmd03.mix/sov06lmd.map	141376	f55b65fd33a53dd03f61b48259d7732d260a4f97170e6dcf0767a61af82403a4
mapsmd03.mix/sov07tmd.map	367021	eacf89952cb6e8eacd762f97895a062e0b6bfa16c8f916b901c1e667def266d4